Final Year Project Documentation

# MediLingo AI

## A Cross-Lingual Medical RAG Chatbot

*Supporting English, Urdu and Roman Urdu*


Presented By

**[YOUR FULL NAME]**

Reg. No: [YOUR REGISTRATION NUMBER]

Supervised By

**[SUPERVISOR NAME]**

[SUPERVISOR DESIGNATION]

Department of [DEPARTMENT NAME]

**[UNIVERSITY NAME], [CITY]**


([START YEAR]–[END YEAR])

# CERTIFICATE OF APPROVAL

**Supervisor:** _____

**[SUPERVISOR NAME]**

[SUPERVISOR DESIGNATION]

Department of [DEPARTMENT NAME]

[UNIVERSITY NAME], [CITY].

**Chairperson:** _____

**[CHAIRPERSON NAME]**

[CHAIRPERSON DESIGNATION]

Department of [DEPARTMENT NAME]

[UNIVERSITY NAME], [CITY].

**Date:** _____

# ACKNOWLEDGEMENT

[Write your acknowledgement here in your own words.]

**[YOUR FULL NAME]**

# ABSTRACT

Pakistan is home to over 230 million people, the majority of whom communicate in Urdu or Roman Urdu as their primary language. When seeking medical guidance outside a clinical setting — to understand a diagnosis, research a medication or evaluate symptoms — this population is largely underserved. The global landscape of AI-powered medical tools is built almost exclusively around English: queries are expected in English, knowledge bases are indexed in English and responses are generated in English. A Urdu-speaking user asking about drug interactions or post-operative care is either met with silence, an inaccurate machine translation, or a response from a general-purpose model that is not anchored in verified clinical knowledge and may fabricate medical details.

**MediLingo AI** is a web-based medical chatbot built on a Retrieval-Augmented Generation (RAG) architecture that natively handles queries in English, Urdu and Roman Urdu. The backend is developed in Python using FastAPI and integrates three core components: a ChromaDB vector store that indexes semantically chunked medical PDF documents, a `sentence-transformers` embedding model (`all-MiniLM-L6-v2`) for dense passage retrieval, and the Groq-hosted LLaMA 3.3-70b large language model for response generation. The React 19 / TypeScript / Vite frontend provides a persistent, session-based chat interface with voice input and output via the Web Speech API.

A central design decision is the *language-prefix system*: the user's chosen language is prepended to every query as a natural-language instruction before the query reaches the LLM, while the embedding pipeline strips this prefix so that vector search operates on the semantic content of the medical question rather than the language tag. Authentication uses a JWT dual-token scheme — short-lived access tokens held in memory and long-lived refresh tokens stored in HTTP-only cookies — with all session history persisted in MongoDB.

The system demonstrates that a multilingual, domain-grounded medical assistant is achievable using lightweight open-source models, closing a practical healthcare information gap for tens of millions of Urdu and Roman Urdu speakers.

# Table of Contents

# List of Figures

# List of Tables

Chapter 1

# SOFTWARE PROJECT MANAGEMENT PLAN (SPMP)

## 1.1 Introduction

Pakistan is home to over 230 million people, and for most of them Urdu or Roman Urdu is the language they actually think in. Yet if someone in Lahore or Karachi wants to look up drug interactions or understand a diagnosis, every halfway-decent AI tool on the internet responds in one language only: English. Symptom checkers, clinical chatbots, question-answering systems — built for English, returning English, assuming English is the default.

That is not a small gap. For a population this size, it is a structural barrier to basic health information. General-purpose AI assistants like ChatGPT can technically respond in Urdu, but they pull their medical knowledge straight from training weights — no verified source, no updated corpus, no way to trace where a claim came from. In a medical setting, a confident but wrong answer is not just unhelpful. It is dangerous.

What the problem needs is a system that combines the language flexibility of modern LLMs with retrieval from a trusted, curated medical knowledge base. Figure 1.1 shows the high-level design of MediLingo AI, which connects a React/Vite frontend to a FastAPI backend running a full RAG pipeline — PDF ingestion, semantic indexing, and multilingual response generation via a hosted large language model.

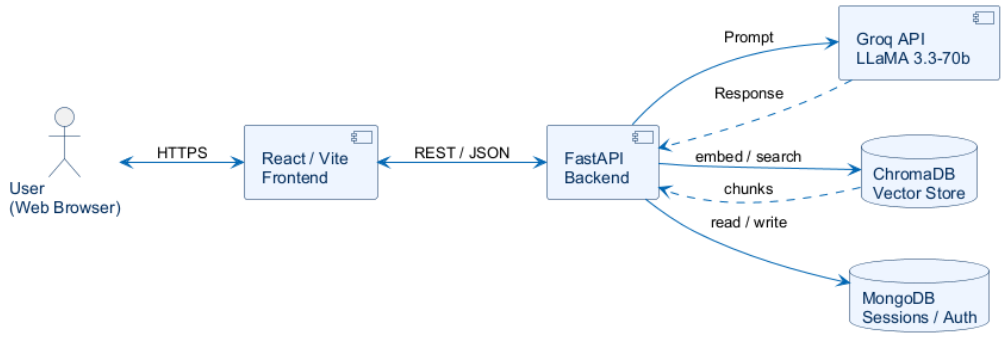



**Figure 1.1:** MediLingo AI — High-Level System Overview

### 1.1.1 Existing Medical Information Systems

#### *Existing Systems With AI Capabilities*

A number of commercial platforms already use AI for medical information delivery. Each one has the same blind spot: they are built around English, and there is no real pathway for someone who wants to type a query in Urdu.

- **Ada Health [1]:** Ada is a conversational symptom checker that walks users through a structured set of questions and returns a ranked differential of probable conditions. Its clinical reasoning engine is proprietary and English-only. It does not accept free-text queries in Urdu and provides no retrieval mechanism against a user-supplied knowledge base.

**[Figure Placeholder]**

*Screenshot of Ada Health symptom checker interface*

**Figure 1.2:** Ada Health — AI-driven symptom checker (English only)

- **Babylon Health:** Babylon's AI triage service operates on a closed clinical model

trained on English-language medical data. It has never been deployed in Pakistan and does not support Urdu as either an input or output language.

**[Figure Placeholder]**

*Screenshot of Babylon Health AI consultation interface*

**Figure 1.3:** Babylon Health — AI triage and health consultation

- **Med-PaLM 2 (Google) [2]:** Med-PaLM 2 is a large language model fine-tuned on medical benchmarks. It scored at expert level on medical licensing exams, which is impressive. But it is a research system, not publicly available, and it operates in English only — so for a Urdu-speaking user, its benchmark scores are essentially irrelevant.
- **WebMD Symptom Checker:** WebMD uses a keyword-driven symptom tool that matches inputs to conditions in a static database. No natural-language generation, no retrieval pipeline, no Urdu support. It is more of a lookup tool than an intelligent assistant.

### *Existing Systems Without Multilingual AI*

On the Pakistani side, the platforms that actually serve this population provide health information without any real AI query capability:

- **Sehat.com.pk and Marham.pk:** These are the most widely used Pakistani health portals. They list doctors, allow appointment booking and publish basic health articles — some of it in Urdu. But neither offers an interactive medical query interface, and there is no generative response system or retrieval from a knowledge

base. Useful for finding a doctor; not useful for asking a clinical question.

- **General-purpose LLMs (e.g., ChatGPT):** Models like ChatGPT do understand Urdu and Roman Urdu reasonably well. The problem is that their medical knowledge comes from training data, not a verified clinical corpus. It cannot be updated, and there is no way to trace a claim back to a source document. That is a meaningful risk in a medical context — the answer might be fluent and confident while being clinically wrong or outdated.
- **Government and hospital websites:** Static health content published by Pakistani hospitals and the Ministry of National Health Services is in English. None of these platforms offer any interactive query interface at all.

### 1.1.2 Project Overview

MediLingo AI is a web-based medical chatbot that takes queries in English, Urdu and Roman Urdu and returns responses drawn from an indexed collection of medical PDF documents. The key distinction from a general-purpose chatbot is that responses are not generated from model memory — they are grounded in passages retrieved from the vector store at query time. The LLM is there to synthesise and explain, not to recall.

The backend runs on FastAPI [3] and handles four things: user authentication and session storage via MongoDB [4] and JWT tokens; PDF ingestion and semantic chunking via `pdfplumber`; dense retrieval via ChromaDB [5] and `sentence-transformers` [6]; and response generation through the Groq-hosted LLaMA 3.3-70b model [7]. The frontend is React 19 with TypeScript and Vite [8], styled with TailwindCSS, and uses the browser's Web Speech API for voice interaction.

The key features of the system are:

- User registration, login and password reset via email OTP
- Multilingual query input in English, Urdu and Roman Urdu
- RAG-grounded responses generated from ingested medical PDFs
- Full chat session management: create, continue, rename and delete sessions

- Persistent message history stored in MongoDB
- Voice input (speech-to-text) and voice output (text-to-speech)
- Admin panel for user management and triggering PDF ingestion
- JWT dual-token authentication with HTTP-only refresh cookies

### 1.1.3 Project Deliverables

- Software Project Management Plan (SPMP)
- Software Requirements Specification (SRS)
- Software Design Description (SDD)
- Software Test Documentation (STD)
- FastAPI backend with integrated RAG engine and ingestion pipeline
- React / TypeScript / Vite frontend with multilingual chat interface
- ChromaDB vector store populated from medical PDF documents
- MongoDB schema for users, sessions and messages
- Docker and Docker Compose configuration for single-command deployment
- GitHub repository with complete version-controlled source code
- Final working demonstration of the cross-lingual RAG flow

## 1.2 Project Organization

### 1.2.1 Software Process Model

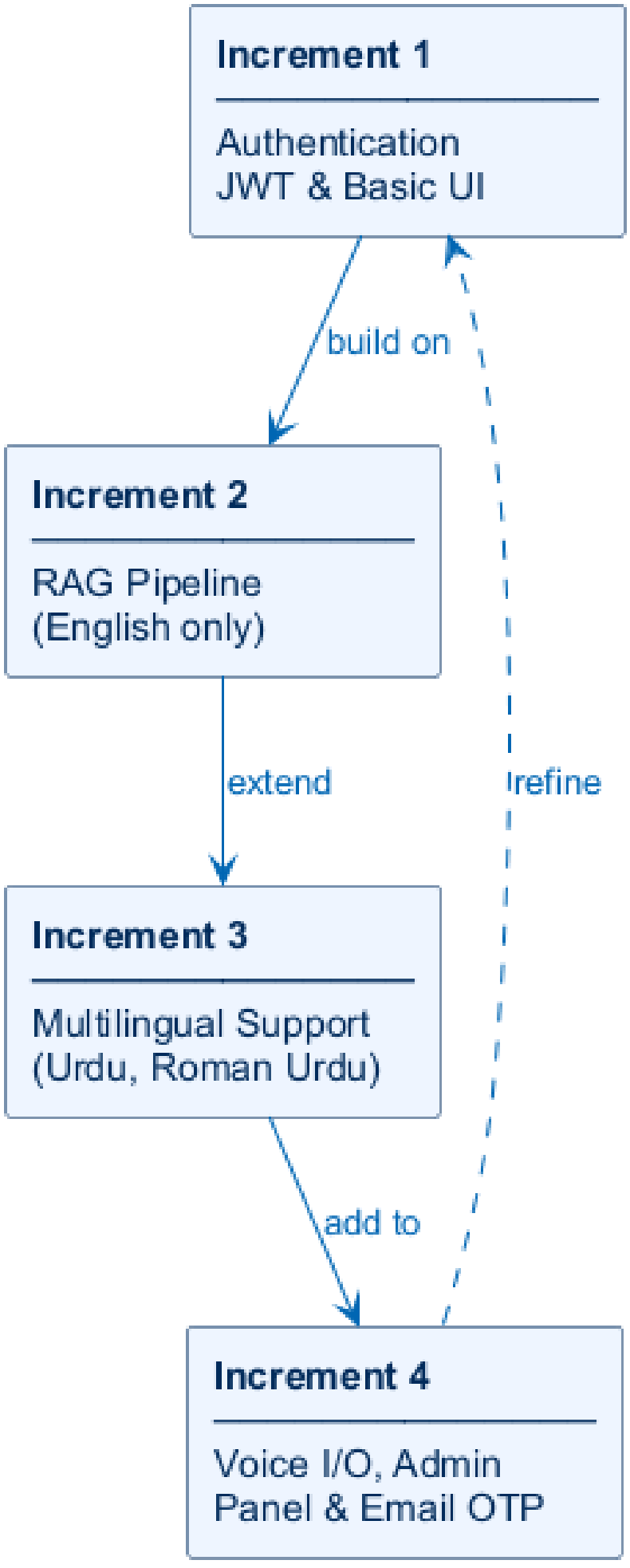
The **Iterative Incremental Model** was chosen for MediLingo AI. The reasoning was straightforward once the system's structure was clear:

- The system breaks down naturally into independent, shippable increments. Authentication and session storage can be built and fully verified before a single line of

RAG code is written. The English-only RAG pipeline can be validated before Urdu support is layered on. Voice features can be added last without touching anything below them. Each increment leaves the system in a working state.

- A RAG pipeline requires empirical tuning — chunk size, overlap length, top-k retrieval count, similarity threshold. These values cannot be derived analytically; they have to be evaluated against real queries. An iterative model makes this feedback loop part of the process rather than something tacked on at the end.
- The language-prefix stripping mechanism was added mid-project, after it became clear that embedding the full prefixed query was degrading retrieval quality. An iterative approach meant this design change could be isolated to one increment, tested and confirmed without breaking anything already shipped.

Figure 1.4 shows the four increments and how each builds on the last.


Increment 1
Authentication
JWT & Basic UI
build on
Increment 2
RAG Pipeline
(English only)
extend
refine
Increment 3
Multilingual Support
(Urdu, Roman Urdu)
add to
Increment 4
Voice I/O, Admin
Panel & Email OTP

### 1.2.2 Roles and Responsibilities

**Developer (Student):**

- Design and implement all FastAPI routers covering authentication (`/auth/*`), session management (`/sessions/*`) and RAG query processing (`/query`, `/retrieve`, `/pipeline/*`)
- Build the RAG ingestion pipeline: PDF extraction with `pdfplumber`, semantic chunking, embedding generation and ChromaDB indexing
- Integrate the Groq API and implement the language-prefix stripping mechanism so that semantic search operates on clean English text regardless of the query language
- Develop the React frontend including the chat page, session sidebar, login, register, profile, admin, forgot-password and reset-password pages
- Implement voice input and output using the Web Speech API through custom hooks
- Set up JWT dual-token authentication with silent refresh on 401 responses
- Configure Docker and Docker Compose for the full stack
- Write all project documentation and prepare diagrams

**Supervisor:**

- Provide architectural guidance on the RAG design and multilingual handling strategy
- Review documentation milestones and provide written feedback
- Clarify requirements and resolve design ambiguities during development

### 1.2.3 Tools and Techniques

**Tools Used:**

- **Python 3.11 and FastAPI [3]** — asynchronous REST API framework with built-in OpenAPI documentation at `/docs`

- **React 19, TypeScript and Vite [8]** — component-based frontend with fast hot-module replacement during development
- **TailwindCSS** — utility-first CSS framework for all UI styling
- **Zustand** — minimal state management for auth tokens and session list
- **MongoDB and Motor [4]** — async NoSQL store for `UserDoc`, `SessionDoc` and `MessageDoc` records
- **ChromaDB [5]** — embedded vector database; persists to `backend/vectorstore/` and supports cosine similarity search
- **sentence-transformers `all-MiniLM-L6-v2` [6]** — 384-dimensional dense embedding model loaded once at startup via the RAG service singleton
- **Groq API (LLaMA 3.3-70b) [7]** — cloud-hosted inference endpoint for final response generation
- **pdfplumber / pypdf** — PDF text extraction for the ingestion pipeline
- **Docker and Docker Compose** — containerised deployment of backend, frontend and MongoDB as a single `docker-compose up` command
- **Git and GitHub** — version control; all increments developed on the same main branch with descriptive commits
- **PlantUML** — UML diagram generation for all SRS and SDD sequence diagrams, domain model, class diagram, use case diagram and Gantt chart
- **VS Code + MiKTeX + LaTeX Workshop** — documentation typesetting for all four documents compiled locally with pdflatex

**Techniques Used:**

- Retrieval-Augmented Generation (RAG) [9] — retrieval from a curated domain corpus before language model generation
- Semantic chunking at 1000 characters with 200-character overlap to preserve sentence

context across chunk boundaries

- Cosine similarity search over 384-dimensional embeddings for passage retrieval
- Language-prefix system: user language instruction prepended to query for the LLM but stripped before embedding to keep retrieval language-neutral
- JWT dual-token authentication: 15-minute access tokens (in-memory only) plus 7-day refresh tokens (HTTP-only cookie) with automatic silent refresh on 401
- RAG singleton lifecycle managed by `rag_service.py` with deferred import of heavy ML libraries (`sentence-transformers`, `chromadb`) to avoid slowing FastAPI startup
- UUID4 string IDs for all MongoDB documents to decouple application identity from the MongoDB `_id` field

## 1.3 Project Management Plan

### 1.3.1 Tasks

The project is organised into six phases executed in iterative sequence:

- Requirements Analysis Phase
- System Design Phase
- Backend Development Phase
- Frontend Development Phase
- RAG Integration and Multilingual Testing Phase
- Testing and Validation Phase

### 1.3.2 Description

The **Requirements Analysis Phase** was really about pinning down what the system needed to do before any code was written. This meant listing out the functional requirements —

registration, multilingual medical query, session management, voice I/O, admin ingestion trigger — and the non-functional ones around latency, token security and browser compatibility for voice features.

During the **System Design Phase**, the three-tier architecture was finalised alongside the MongoDB document schema, the ChromaDB collection design and the RAG pipeline dataflow. The language-prefix stripping contract between frontend and backend was one of the more important design decisions made here. UML artefacts — use-case diagrams, system sequence diagrams, domain model and class diagram — came out of this phase.

The **Backend Development Phase** is where the actual implementation began. All FastAPI routers were built, along with `RAGQueryEngine`, the ingestion pipeline class `RAGIngestionPipeline`, the Motor-based database layer and the JWT auth service. The Groq API integration and the async lifespan startup sequence were also completed here.

The **Frontend Development Phase** covered all the React pages and custom hooks: landing page, chat interface with session sidebar, login and registration flows, profile page, forgot-password and reset-password pages, admin panel, and the `useVoiceInput` / `useVoiceOutput` hooks. Vite's proxy was configured so API routes forwarded to `localhost:8000` during development — this was important for getting cookies to work correctly without fighting CORS.

The **RAG Integration and Multilingual Testing Phase** connected the live frontend to the backend and ran queries in all three languages. This was the phase where the language-prefix mechanism was validated — the test was simple: switching language should not change which documents get retrieved, only how the response is worded. It passed.

The **Testing and Validation Phase** covered functional testing of every use case: JWT token lifecycle, RAG retrieval quality, voice API in Chrome and Edge, and admin workflows. Issues found here were fixed before final submission.

### 1.3.3 Project Plan

Figure 1.5 shows the project plan as a Gantt chart. The backend and frontend development phases overlap intentionally — waiting for the backend to be completely finished before touching the frontend would have stretched the timeline by weeks. The RAG integration phase that follows both is where the two halves are connected and tested end-to-end.

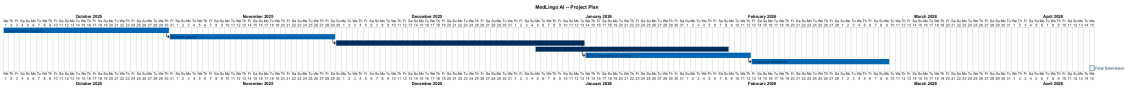



**Figure 1.5:** Project Management Plan — Gantt Chart

### 1.3.4 Project Timeline

Figure 1.6 marks the key milestones from requirements analysis to final submission. The documentation-heavy first half and implementation-heavy second half are not a coincidence — writing down what the system needs to do before writing the code for it saves significant rework later. That principle drove the schedule.

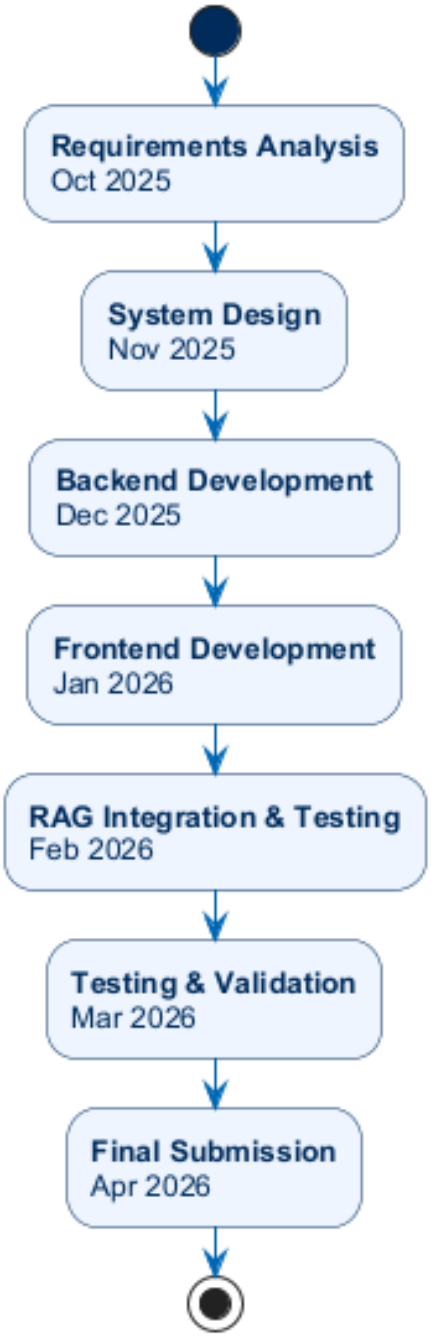



**Figure 1.6:** MediLingo AI — Project Timeline

Chapter 2

# SOFTWARE REQUIREMENT SPECIFICATIONS (SRS)

Getting requirements right before writing a single line of code is one of those lessons that takes most developers a bad experience to learn. This chapter documents what MediLingo AI is actually supposed to do: the problem motivating it, the boundaries it works within, and the complete set of functional and non-functional requirements. Use cases and the domain model sit at the end, building on everything before them. The system design in Chapter 3 should be fully traceable back to this chapter.

## 2.1 Introduction

### 2.1.1 Problem Statement

Pakistan has one of the largest Urdu-speaking populations in the world, yet there is no publicly accessible, AI-driven medical tool that accepts queries in Urdu or Roman Urdu and grounds its answers in verified medical documents. The gap has two distinct layers.

The first is language. Every AI medical tool of any quality is built for English. They cannot process a query written in Urdu, and even when a user translates their question manually, there is no mechanism to return a response in their native language. The second layer is reliability. General-purpose LLMs that do support Urdu produce answers from training weights, not from a curated clinical corpus. In healthcare, a confident but wrong answer is not just unhelpful — it carries real risk. Medical facts, dosage details and symptom descriptions need to come from somewhere traceable, not from statistical patterns baked into model parameters.

MediLingo AI addresses both. It accepts queries in English, Urdu and Roman Urdu, and every response is grounded in passages retrieved from an indexed medical knowledge base at query time — not recalled from model memory.

### 2.1.2 Motivation

The case for building this came down to three things that kept coming back during the requirements phase.

- **Language accessibility:** Forcing an Urdu speaker to translate their medical question before they can get a useful answer is not a minor inconvenience. For a lot of people it is a genuine barrier. Removing the translation step lowers the cost of accessing health information for tens of millions of Urdu speakers. That felt like a concrete, achievable goal rather than an abstract one.
- **Factual reliability:** The hallucination problem is well-known. RAG is not a complete solution, but it meaningfully narrows the gap by tying every response to specific retrieved passages from a known knowledge base. At minimum, it makes the system's outputs traceable to a source document — something a standard LLM pulling from training memory alone cannot do.
- **Practical feasibility:** The components needed for this — open-source embedding models, a cloud-hosted LLM, a local vector database — are mature enough now that a working system is achievable without proprietary data or specialised hardware. That was not obviously true two or three years ago.

### 2.1.3 Objectives

The primary objectives of the MediLingo AI project are:

- Build a web-based medical chatbot accepting queries in English, Urdu and Roman Urdu, returning responses in the same language as the query
- Ground every response in verified medical PDF documents through a RAG pipeline — the LLM synthesises and explains, but does not recall clinical facts from memory

- Give users a persistent, session-based chat interface so that separate medical conversations can be maintained and revisited over time
- Implement JWT dual-token authentication with HTTP-only refresh cookies and silent renewal so the user's session does not interrupt them
- Integrate voice input and output through the Web Speech API, making the system accessible to users who prefer speech interaction
- Provide an admin interface for user account management and knowledge base re-ingestion from PDF files placed in the server datasets directory

### 2.1.4 Scope

**In scope:**

- Web application accessible from any modern browser
- Query input and response output in English, Urdu and Roman Urdu
- RAG pipeline using locally ingested medical PDF documents as the knowledge base
- User registration, login, logout and password reset
- Chat session creation, continuation, deletion and history viewing
- Voice input (speech-to-text) and voice output (text-to-speech)
- Admin panel for user management and PDF ingestion trigger
- Containerised deployment via Docker and Docker Compose

**Out of scope:**

- Native mobile application (iOS or Android)
- Diagnostic conclusions or prescriptive medical advice — the system provides information only, not clinical diagnosis
- Real-time consultation with a licensed medical professional

- Support for languages other than English, Urdu and Roman Urdu
- Integration with external electronic health record (EHR) systems

### 2.1.5 Constraints

- A valid `GROQ_API_KEY` environment variable is required; the backend starts successfully without it but every query returns an error until it is set
- Voice input and output depend on the Web Speech API, which has full support only in Chromium-based browsers (Chrome, Edge) and partial support in Firefox; Safari is unsupported for voice input
- ChromaDB is used in embedded mode, meaning it is co-located with the backend process and does not support distributed or replicated vector storage out of the box
- The `sentence-transformers` model (`all-MiniLM-L6-v2`) is loaded fully into RAM on startup; the server requires a minimum of 512 MB of available memory for the ML components alone
- Medical knowledge is limited to the content of PDF files placed in `backend/datasets/` and ingested through the admin pipeline; the system cannot answer questions on topics not covered by those documents
- The language-prefix mechanism relies on the Groq-hosted LLaMA 3.3-70b model's built-in multilingual capability; the quality of Urdu and Roman Urdu responses depends on the model's training data and may vary by medical sub-domain

### 2.1.6 Assumptions

- All medical PDF documents provided for ingestion are written in English; the embedding model operates on English text, and translating source documents is outside the scope of the system
- Users have a stable internet connection sufficient for REST API calls and Groq inference requests

- The Groq API service remains available during system operation; no fallback LLM is configured
- Users interact with the system through a web browser on a desktop or laptop device; mobile browser support is functional but not the primary design target
- The admin user is technically familiar enough to place PDF files in the correct server directory and invoke the ingestion endpoint

### 2.1.7 System Workflow Diagram

Figure 2.1 shows what actually happens to a query from the moment the user hits submit to the moment the response lands on screen. The language prefix gets stripped early in the process so that vector search always runs against clean medical text — this is the key design detail that makes multilingual retrieval work correctly.

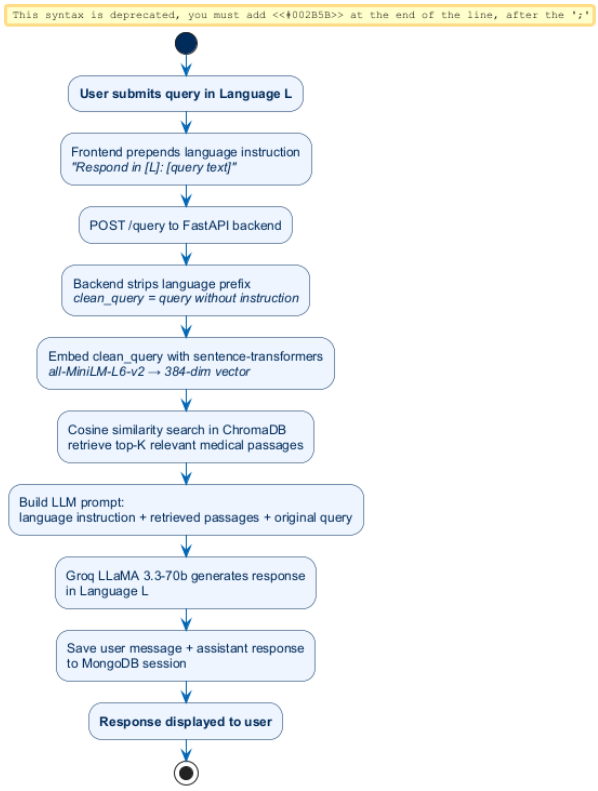



**Figure 2.1:** MediLingo AI — End-to-End System Workflow for Multilingual Medical Query

## 2.2 Overall Description

### 2.2.1 Product Perspective

MediLingo AI is a self-contained web application. It does not plug into any larger platform and depends on only three external services: the Groq API for LLM inference, MongoDB for persistent storage, and ChromaDB for document retrieval. All three can run on the same host as the FastAPI backend or on separate infrastructure without any changes to the application code. During development, a Vite proxy forwards all API traffic from the browser to `localhost:8000`, which means browser cookies work without fighting CORS — one fewer thing to debug during development.

### 2.2.2 User Characteristics

**Regular User:** Someone looking for medical information — could be fluent in English, Urdu or Roman Urdu, or switching between all three. The interface requires basic internet literacy and nothing more. Users should not need to understand anything about RAG or how the backend works. Voice support is there specifically for people who have difficulty typing or just prefer speaking.

**Admin User:** Someone with enough technical familiarity to place PDF files in the correct server directory and trigger the ingestion endpoint from the admin panel. They authenticate through a dedicated admin login route that verifies the role field in the JWT payload before granting access to admin-only endpoints.

## 2.3 Specific Requirements

### 2.3.1 External Interface Requirements

**User Interface:** A React SPA delivered in the browser. The interface includes a landing page, auth pages (login, register, forgot password, reset password), a chat page with a collapsible session sidebar, a profile page and an admin panel. No plugin or browser extension is required.

**Hardware Interface:** A standard desktop or laptop with a microphone for voice input, and speakers or headphones for voice output. No specialised hardware needed.

**Software Interface:** The backend exposes a documented REST API at `/docs` (Swagger

UI). The frontend uses this interface exclusively. The backend communicates with MongoDB via Motor and with ChromaDB via its Python client.

**Communication Interface:** All frontend-to-backend communication is HTTP/HTTPS with JSON bodies. Auth credentials go as a Bearer token in the `Authorization` header. Refresh tokens are transmitted as HTTP-only cookies.

### 2.3.2 Functional Requirements

**Table 2.1:** Functional Requirements of MediLingo AI

| ID | Requirement | Description |
|---|---|---|
| **FR-01** | User Registration | The system shall allow a new user to create an account using a display name, email address and password. Duplicate email addresses must be rejected. |
| **FR-02** | User Login | The system shall authenticate a registered user with their email and password, issue a JWT access token and set an HTTP-only JWT refresh cookie. |
| **FR-03** | Silent Token Refresh | The frontend shall automatically call `POST /auth/refresh` when the backend returns HTTP 401, retry the original request with the new access token, and redirect to login only if the refresh token has also expired. |
| **FR-04** | User Logout | The system shall invalidate the refresh cookie and clear in-memory auth state on explicit logout. |

| ID | Requirement | Description |
|---|---|---|
| **FR-05** | Password Reset via Email OTP | The system shall allow a registered user to reset their password by requesting a one-time passcode sent to their registered email address. |
| **FR-06** | Multilingual Query Submission | The system shall accept medical queries in English, Urdu and Roman Urdu and return a response in the same language as the query. |
| **FR-07** | RAG-Grounded Response Generation | Every query response shall be generated by first retrieving the top-K semantically relevant passages from the ChromaDB vector store and including them in the LLM prompt. |
| **FR-08** | Language-Prefix Stripping | The backend shall strip the language instruction prefix from the query text before computing the embedding, so that vector search operates on the medical content alone. |
| **FR-09** | Chat Session Creation | An authenticated user shall be able to start a new chat session. The session shall be saved to MongoDB and appear in the session sidebar. |
| **FR-10** | Chat Session Management | An authenticated user shall be able to view, continue and delete any of their saved chat sessions. All messages within a deleted session shall be removed. |
| **FR-11** | Message Persistence | Every user query and assistant response within a session shall be saved as a `MessageDoc` in MongoDB so that session history survives page reload. |

| ID | Requirement | Description |
|---|---|---|
| **FR-12** | Voice Input | The system shall provide a microphone button that activates the browser's Web Speech API to transcribe speech into the query input field. |
| **FR-13** | Voice Output | The system shall read assistant responses aloud using the browser's SpeechSynthesis API after each response is received. |
| **FR-14** | User Profile Management | An authenticated user shall be able to view and update their display name and email address from the profile page. |
| **FR-15** | Admin: User Management | An admin user shall be able to view all registered accounts and activate or deactivate individual users. |
| **FR-16** | Admin: PDF Ingestion | An admin user shall be able to trigger a synchronous or background ingestion of PDF files from `backend/datasets/` into the ChromaDB vector store via the admin panel. |

### 2.3.3 Non-Functional Requirements

**Table 2.2:** Non-Functional Requirements of MediLingo AI

| ID | Category | Requirement |
|---|---|---|
| **NFR-01** | Performance | End-to-end query response time (from submission to display) shall not exceed 15 seconds under normal Groq API latency conditions. |

| ID | Category | Requirement |
|---|---|---|
| **NFR-02** | Security | User passwords shall be stored as bcrypt hashes. Access tokens shall never be persisted to localStorage or sessionStorage. Refresh tokens shall be stored in HTTP-only, Secure cookies only. |
| **NFR-03** | Reliability | The RAG engine and ingestion pipeline shall be initialised as a singleton at backend startup. If initialisation fails, the backend shall still start and return a descriptive error on query endpoints rather than crashing entirely. |
| **NFR-04** | Availability | The system shall be deployable as a set of Docker containers that can be started with a single `docker-compose up` command, minimising manual deployment steps. |
| **NFR-05** | Maintainability | Backend logic shall be separated into distinct FastAPI routers (`auth`, `sessions`, `query`). Frontend business logic shall be isolated in custom React hooks, separate from UI components. |
| **NFR-06** | Portability | The application shall run on any machine with Docker installed, regardless of the underlying operating system. |
| **NFR-07** | Browser Compatibility | The web application shall function correctly on Chrome 120+, Edge 120+ and Firefox 121+. Voice features shall degrade gracefully (button hidden or disabled) on unsupported browsers. |

| ID | Category | Requirement |
|---|---|---|
| **NFR-08** | Usability | The language selector, voice button and session sidebar shall be visible and operable without reading any documentation. The interface shall be fully functional on a 1280×720 desktop viewport. |

## 2.4 Software Product Features

MediLingo AI exposes the following top-level product features:

- **Cross-Lingual Medical Chat** — queries and responses in English, Urdu or Roman Urdu, with language routing handled automatically by the prefix mechanism in the frontend and backend
- **RAG-Grounded Answers** — each response is generated from retrieved passages rather than from the LLM's memory; medical content is always traceable to a source document
- **Persistent Session History** — users maintain multiple named conversations and can return to any of them at any time; message history survives page reloads because it is stored in MongoDB, not in browser memory
- **Voice Interaction** — speech-to-text for query input and text-to-speech for response playback; both use the browser's built-in Web Speech API with no additional plugin required
- **Secure Authentication** — 15-minute access tokens kept in memory, 7-day refresh tokens in HTTP-only cookies, automatic silent renewal so the user's session does not get interrupted
- **Admin Knowledge Management** — administrators can expand or refresh the medical knowledge base by triggering a new PDF ingestion run through the admin panel

- **Email-Based Password Recovery** — users can regain access via a time-limited one-time passcode sent to their registered email address

## 2.5 Software System Attributes

### 2.5.1 Reliability

The RAG engine and ingestion pipeline are initialised as module-level singletons in `rag_service.py`, loaded once when the FastAPI server starts. If the Groq API is temporarily unavailable, the backend logs the error and returns a structured HTTP 500 rather than taking down the server process. MongoDB is handled the same way. The guiding principle was straightforward: one broken external dependency should not kill the whole application, and a structured error message is almost always more useful than a crash.

### 2.5.2 Availability

The system is packaged with Docker Compose. A single `docker-compose up` command starts the FastAPI backend, the React frontend and MongoDB together. Once running, the application is available at `localhost:5173` (frontend) and `localhost:8000` (API). No further configuration is required beyond setting the environment variables described in the CLAUDE.md file.

### 2.5.3 Security

Passwords go through bcrypt before they ever touch the database — there is no plaintext stored anywhere in the system. Access tokens live exclusively in Zustand in-memory state and disappear when the browser tab closes. Refresh tokens go into HTTP-only cookies that JavaScript cannot read, which closes off the most common XSS token-theft vector. CORS is configured in `main.py` to restrict cross-origin requests to the deployed frontend origin; wildcard origins are not used.

### 2.5.4 Maintainability

The backend splits cleanly into independent FastAPI routers, one file each for auth, sessions and query/pipeline. Adding a new API surface means writing one router file and one `app.include_router()` call — nothing else changes. On the frontend, all data-fetching and auth logic lives in custom hooks (`useAuth`, `useChat`, `useSessions`), keeping UI components stateless. That separation makes it much easier to test business logic independently of the rendering layer.

### 2.5.5 Portability

Everything runs inside Docker containers. The host machine needs Docker Engine and nothing else. Python 3.11 and Node 20 are fixed in the build image specifications so that a build on one machine produces the same result as a build on any other — the "works on my machine" problem is eliminated by design.

### 2.5.6 Performance

Heavy ML imports — `sentence-transformers` and `chromadb` — are deferred inside `init_rag()` and loaded once at startup rather than at module import time. This keeps FastAPI's startup fast and avoids reloading the embedding model on every request. The `all-MiniLM-L6-v2` model produces 384-dimensional embeddings; at that dimensionality, a ChromaDB cosine search across several thousand chunks runs in well under 100 ms on a standard CPU. That leaves the Groq API call as the primary latency variable, not the retrieval step.

## 2.6 Database Requirements

MediLingo AI uses two persistence layers.

**MongoDB** stores three collections:

- `users` — one document per registered account, containing the UUID4 string ID, display name, email, bcrypt password hash, role and active flag
- `sessions` — one document per chat session, containing the session ID, the owning

user's ID and a display title

- `messages` — one document per message (user query or assistant response), containing the parent session ID, the message role (`user` or `assistant`), the content text, the language code and a timestamp

**ChromaDB** stores one collection (`medical_documents`):

- Each record holds the chunk text, its 384-dimensional embedding vector, and metadata including the source filename and chunk index
- The collection is populated by the ingestion pipeline and queried at inference time using cosine similarity
- The collection persists to `backend/vectorstore/` on the host filesystem

## 2.7 Use Cases

### 2.7.1 Use Cases List

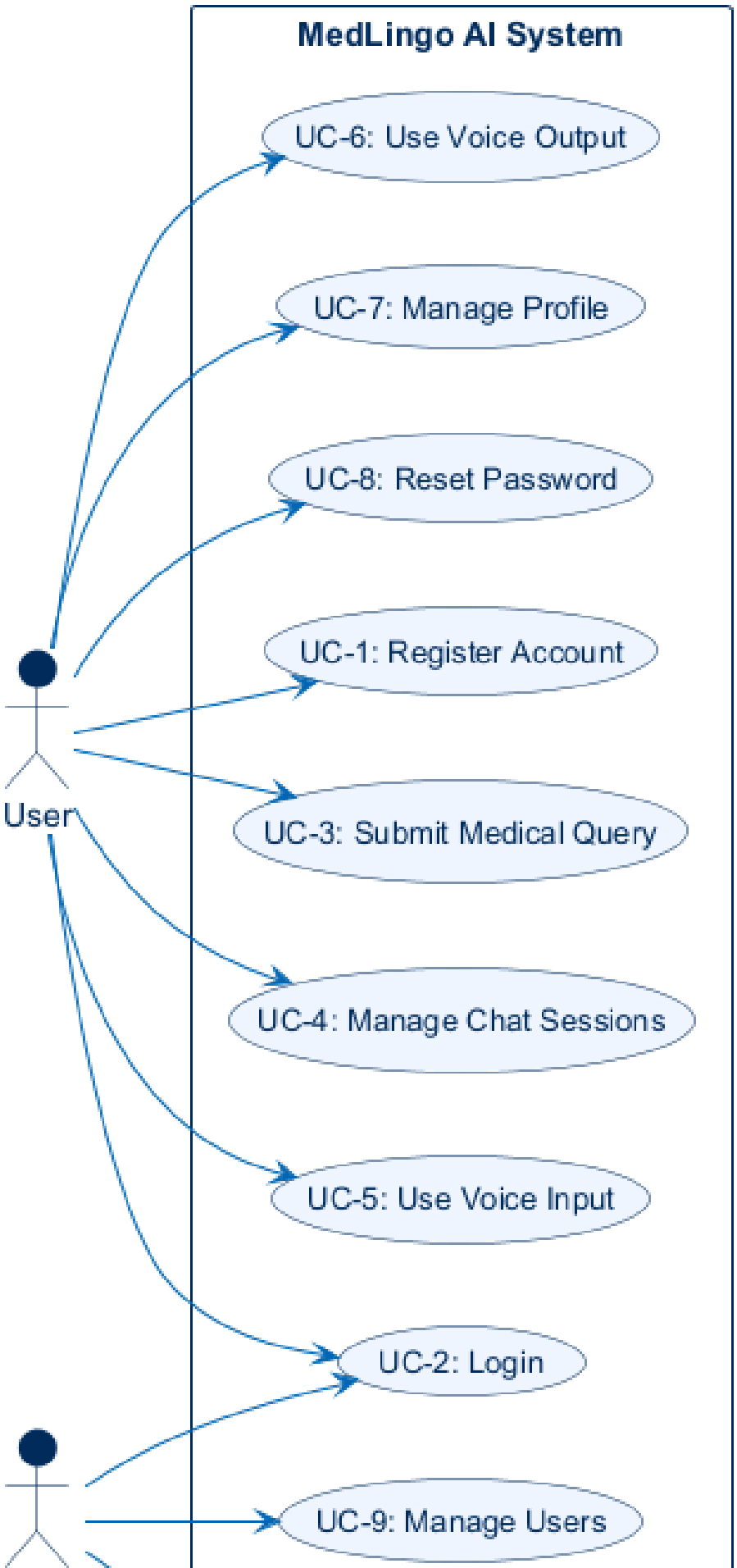
**Table 2.3:** User Use Cases

| UC ID | Use Case Name | Brief Description |
|---|---|---|
| **UC-1** | Register User Account | Create a new MediLingo AI account with name, email and password. |
| **UC-2** | Login | Authenticate an existing account and receive JWT tokens. |
| **UC-3** | Submit Medical Query | Send a multilingual medical question and receive a RAG-grounded response. |
| **UC-4** | Manage Chat Sessions | Create, continue, view and delete persistent chat sessions. |
| **UC-5** | Use Voice Input | Dictate a query via microphone using the Web Speech API. |
| **UC-6** | Use Voice Output | Have an assistant response read aloud via text-to-speech. |
| **UC-7** | Manage Profile | View and update display name and email address. |
| **UC-8** | Reset Password | Recover account access via an email one-time passcode. |

**Table 2.4:** Admin Use Cases

| UC ID | Use Case Name | Brief Description |
|---|---|---|
| **UC-9** | Manage Users | View all accounts; activate or deactivate individual users. |
| **UC-10** | Ingest Medical Documents | Trigger PDF extraction, chunking, embedding and ChromaDB indexing. |

### 2.7.2 Use Case Diagram



### 2.7.3 Use Cases Description

**Table 2.5:** UC-1: Register User Account

| UC-1: Register User Account | |
|---|---|
| **Use Case ID** | UC-1 |
| **Actor** | Unregistered User |
| **Description** | A new user creates a MediLingo AI account by supplying a display name, email address and password. |
| **Preconditions** | The provided email address is not already associated with an existing account. |
| **Main Flow** | 1. User navigates to the Register page.<br>2. User enters display name, email address and password.<br>3. System validates that the email is not already in use.<br>4. System hashes the password with bcrypt and creates a UserDoc in MongoDB.<br>5. System issues a 15-minute access token and sets a 7-day HTTP-only refresh cookie.<br>6. User is redirected to the chat interface. |
| **Alternate Flow** | **A1** – Email already registered: system returns HTTP 400 and the user corrects the email field.<br>**A2** – Password below minimum length: the form is rejected client-side before submission. |
| **Postconditions** | A UserDoc exists in the `users` collection. The user is authenticated with valid JWT tokens. |

**Table 2.6:** UC-2: Login

| **UC-2: Login** | |
|---|---|
| **Use Case ID** | UC-2 |
| **Actor** | Registered User / Admin User |
| **Description** | An existing user authenticates with their email and password to receive JWT tokens. |
| **Preconditions** | A valid account exists for the provided email. The account is active. |
| **Main Flow** | 1. User navigates to the Login page.<br>2. User enters email address and password.<br>3. System verifies the bcrypt hash against the stored value.<br>4. System issues a new access token and sets a new refresh cookie.<br>5. User is redirected to the chat interface. |
| **Alternate Flow** | **A1** – Incorrect credentials: system returns HTTP 401 and the user is prompted to retry.<br>**A2** – Account deactivated: system returns HTTP 403 with an appropriate error message. |
| **Postconditions** | The user's access token is held in Zustand memory. A refresh cookie is set in the browser. |

**Table 2.7:** UC-3: Submit Medical Query

| **UC-3: Submit Medical Query** | |
|---|---|
| **Use Case ID** | UC-3 |
| **Actor** | Authenticated User |
| **Description** | The user submits a medical question in their chosen language and receives a RAG-grounded response in the same language. |
| **Preconditions** | User is authenticated. The RAG engine is initialised. At least one document has been ingested into ChromaDB. |
| **Main Flow** | 1. User selects a language (English / Urdu / Roman Urdu) and types a query.<br>2. Frontend prepends the language instruction to the query text.<br>3. Frontend sends POST `/query` with the prefixed query and session ID.<br>4. Backend strips the language prefix; the clean query is embedded with `all-MiniLM-L6-v2`.<br>5. Backend performs cosine similarity search in ChromaDB and retrieves the top-K passages.<br>6. Backend builds a prompt containing the language instruction, retrieved passages and query.<br>7. Groq LLaMA 3.3-70b generates a response in the selected language.<br>8. Backend saves user message and assistant response as MessageDocs in MongoDB.<br>9. Response is returned to the frontend and displayed in the chat interface. |
| **Alternate Flow** | **A1** – Groq API unavailable: system returns HTTP 500 with "Service temporarily unavailable".<br>**A2** – No relevant passages found: LLM is prompted to indicate insufficient information in the knowledge base.<br>**A3** – Access token expired: `authenticatedFetch` silently calls `/auth/refresh` and retries. |
| | The query and response are saved to the session. The chat view |

**Table 2.8:** UC-4: Manage Chat Sessions

| UC-4: Manage Chat Sessions | |
|---|---|
| **Use Case ID** | UC-4 |
| **Actor** | Authenticated User |
| **Description** | The user creates, switches between and deletes named chat sessions via the session sidebar. |
| **Preconditions** | User is authenticated. |
| **Main Flow** | 1. User clicks “New Session” in the sidebar; a new SessionDoc is created in MongoDB.<br>2. User clicks an existing session to load its message history.<br>3. User deletes a session; the SessionDoc and all child MessageDocs are removed. |
| **Alternate Flow** | **A1** – Network error during session creation: the sidebar shows an error and the session is not added. |
| **Postconditions** | The selected session is active and its messages are loaded in the chat area. |

**Table 2.9:** UC-5: Use Voice Input

| **UC-5: Use Voice Input** | |
|---|---|
| **Use Case ID** | UC-5 |
| **Actor** | Authenticated User |
| **Description** | The user dictates their query via the microphone button; the transcript populates the query input field. |
| **Preconditions** | User is authenticated. Browser supports the Web Speech API. Microphone permission has been granted. |
| **Main Flow** | 1. User clicks the microphone button in the chat interface.<br>2. Browser requests microphone permission if not already granted.<br>3. Speech recognition starts; the user speaks their query.<br>4. Transcript text is placed in the query input field.<br>5. User submits the query normally (UC-3). |
| **Alternate Flow** | **A1** – Microphone permission denied: an error alert is shown and recognition does not start.<br>**A2** – Browser does not support Web Speech API: the microphone button is hidden. |
| **Postconditions** | Query input field contains the transcribed text, ready for submission. |

**Table 2.10:** UC-6: Use Voice Output

| UC-6: Use Voice Output | |
|---|---|
| **Use Case ID** | UC-6 |
| **Actor** | Authenticated User |
| **Description** | After an assistant response is displayed, the system reads it aloud in the user's selected language using the browser's SpeechSynthesis API. |
| **Preconditions** | A response has been received and rendered. Browser supports SpeechSynthesis. |
| **Main Flow** | 1. Assistant response is received by the frontend.<br>2. `useVoiceOutput` hook passes the response text to the SpeechSynthesis API.<br>3. Browser reads the text aloud in the configured language voice. |
| **Alternate Flow** | **A1** – SpeechSynthesis not available: response is displayed as text only; no error is shown. |
| **Postconditions** | The response text has been read aloud or silently displayed if TTS is unavailable. |

**Table 2.11:** UC-7: Manage Profile

| UC-7: Manage Profile | |
|---|---|
| **Use Case ID** | UC-7 |
| **Actor** | Authenticated User |
| **Description** | The user views and updates their display name or email address from the profile page. |
| **Preconditions** | User is authenticated. |
| **Main Flow** | 1. User navigates to the Profile page.<br>2. Current display name and email are pre-filled in the form.<br>3. User edits one or both fields and clicks Save.<br>4. System updates the UserDoc in MongoDB and returns the updated values. |
| **Alternate Flow** | **A1** – New email already taken: system returns HTTP 400 and the user corrects input. |
| **Postconditions** | The UserDoc in MongoDB reflects the updated display name or email. |

**Table 2.12:** UC-8: Reset Password

| **UC-8: Reset Password** | |
|---|---|
| **Use Case ID** | UC-8 |
| **Actor** | Registered User |
| **Description** | A user who has forgotten their password recovers account access via a one-time passcode sent to their registered email address. |
| **Preconditions** | An account exists for the email address provided. |
| **Main Flow** | 1. User navigates to Forgot Password and enters their registered email.<br>2. System generates a time-limited OTP and sends it via the email service.<br>3. User enters the OTP on the Reset Password page.<br>4. User sets and confirms a new password.<br>5. System hashes the new password and updates the UserDoc.<br>6. User is redirected to the Login page. |
| **Alternate Flow** | **A1** – Email not registered: system returns HTTP 404; no OTP is sent.<br>**A2** – OTP expired: user must restart the process from step 1.<br>**A3** – Passwords do not match: form is rejected client-side before submission. |
| **Postconditions** | The user's password hash in MongoDB is updated. The user can log in with the new password. |

**Table 2.13:** UC-9: Manage Users (Admin)

| **UC-9: Manage Users (Admin)** | |
|---|---|
| **Use Case ID** | UC-9 |
| **Actor** | Admin User |
| **Description** | An admin views the full user list and can activate or deactivate individual accounts. |
| **Preconditions** | Admin is authenticated via the admin login route. |
| **Main Flow** | 1. Admin navigates to the Admin Panel.<br>2. System displays a paginated list of all registered users with their status.<br>3. Admin clicks the toggle next to a user to activate or deactivate their account.<br>4. System updates the `is_active` flag in the UserDoc. |
| **Alternate Flow** | **A1** – Admin attempts to deactivate their own account: system returns an error to prevent lockout. |
| **Postconditions** | The targeted user's active status is updated. If deactivated, their next login attempt returns HTTP 403. |

**Table 2.14:** UC-10: Ingest Medical Documents (Admin)

| **UC-10: Ingest Medical Documents (Admin)** | |
|---|---|
| **Use Case ID** | UC-10 |
| **Actor** | Admin User |
| **Description** | An admin triggers the RAG ingestion pipeline to extract, chunk, embed and index PDF files from the datasets directory into ChromaDB. |
| **Preconditions** | Admin is authenticated. One or more PDF files are present in `backend/datasets/`. |
| **Main Flow** | 1. Admin navigates to the Admin Panel and clicks "Ingest Documents".<br>2. Frontend sends POST `/pipeline/ingest-sync`.<br>3. Backend reads all PDF files from `backend/datasets/` using `pdfplumber`.<br>4. Extracted text is split into 1000-character chunks with 200-character overlap.<br>5. Each chunk is embedded with `all-MiniLM-L6-v2`.<br>6. Embeddings and metadata are stored in the `medical_documents` ChromaDB collection.<br>7. System returns a summary of ingested documents and chunk count. |
| **Alternate Flow** | **A1** – No PDF files found: system returns HTTP 400 with "No documents found in datasets directory".<br>**A2** – PDF is password-protected or corrupted: the file is skipped and a warning is included in the response. |
| **Postconditions** | ChromaDB contains embeddings for all successfully processed chunks. Subsequent queries can retrieve passages from the newly ingested documents. |

## 2.8 System Sequence Diagrams

A System Sequence Diagram (SSD) shows the interaction between an actor and the system boundary for a given use case. The system is a black box at this level of abstraction — exactly what the SRS perspective calls for. The internals — how FastAPI coordinates the RAG engine, ChromaDB and MongoDB to produce a response — are covered in the detailed sequence diagrams of Chapter 3.

### 2.8.1 SSD-1: Register User Account

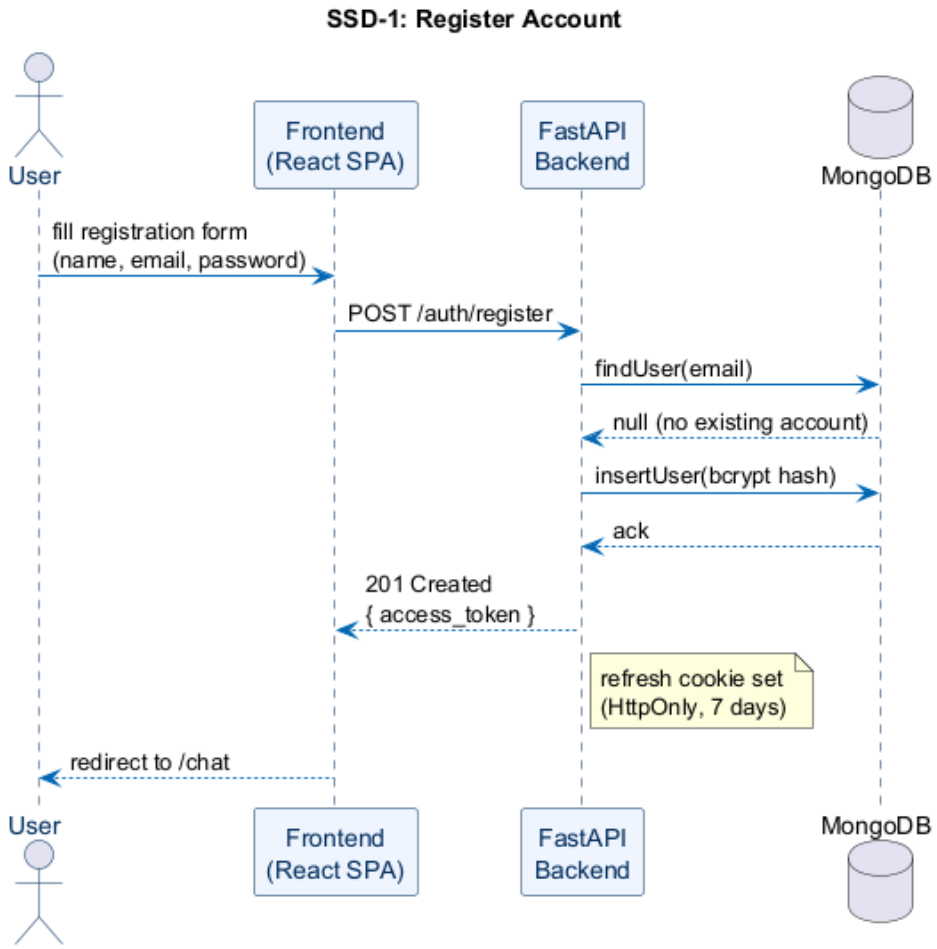



**Figure 2.3:** SSD-1: Register User Account

### 2.8.2 SSD-2: Login

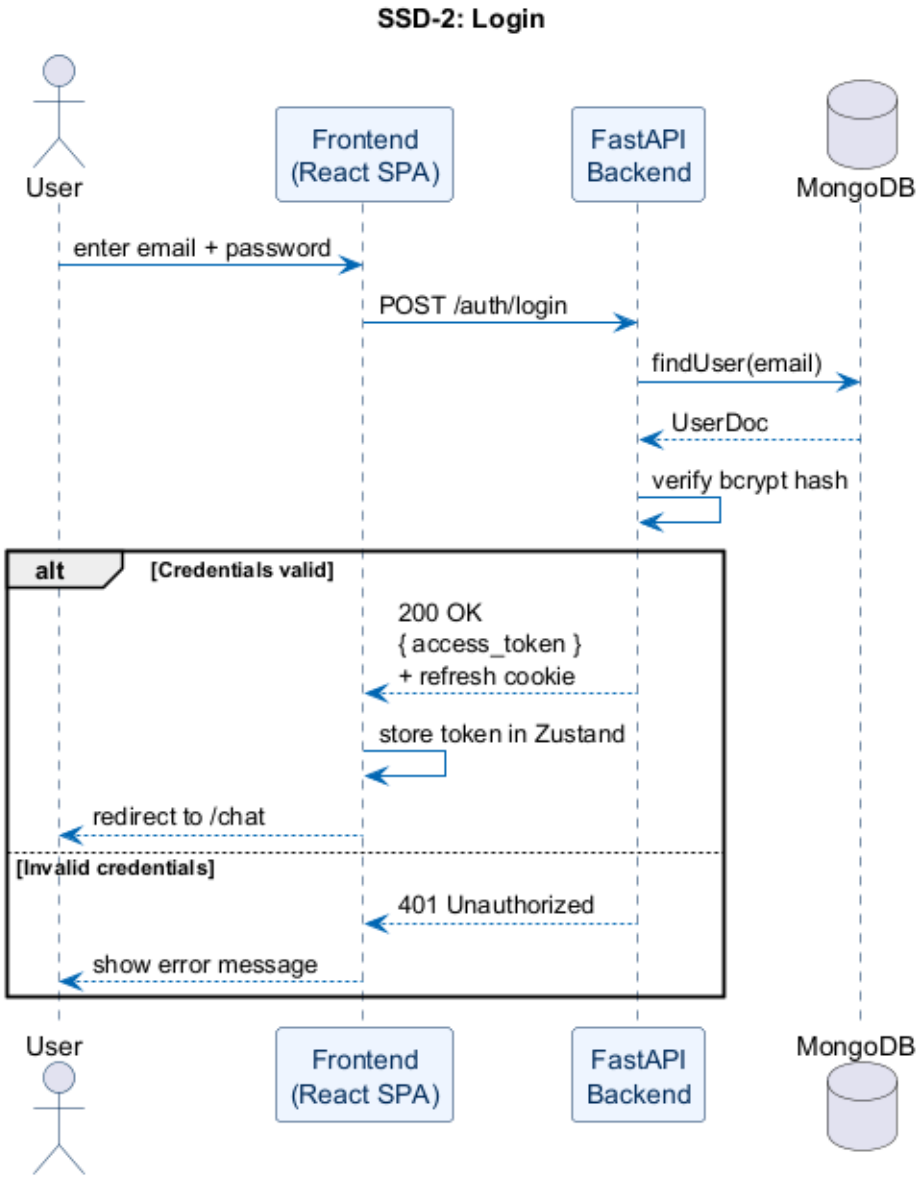



**Figure 2.4:** SSD-2: Login

### 2.8.3 SSD-3: Submit Medical Query

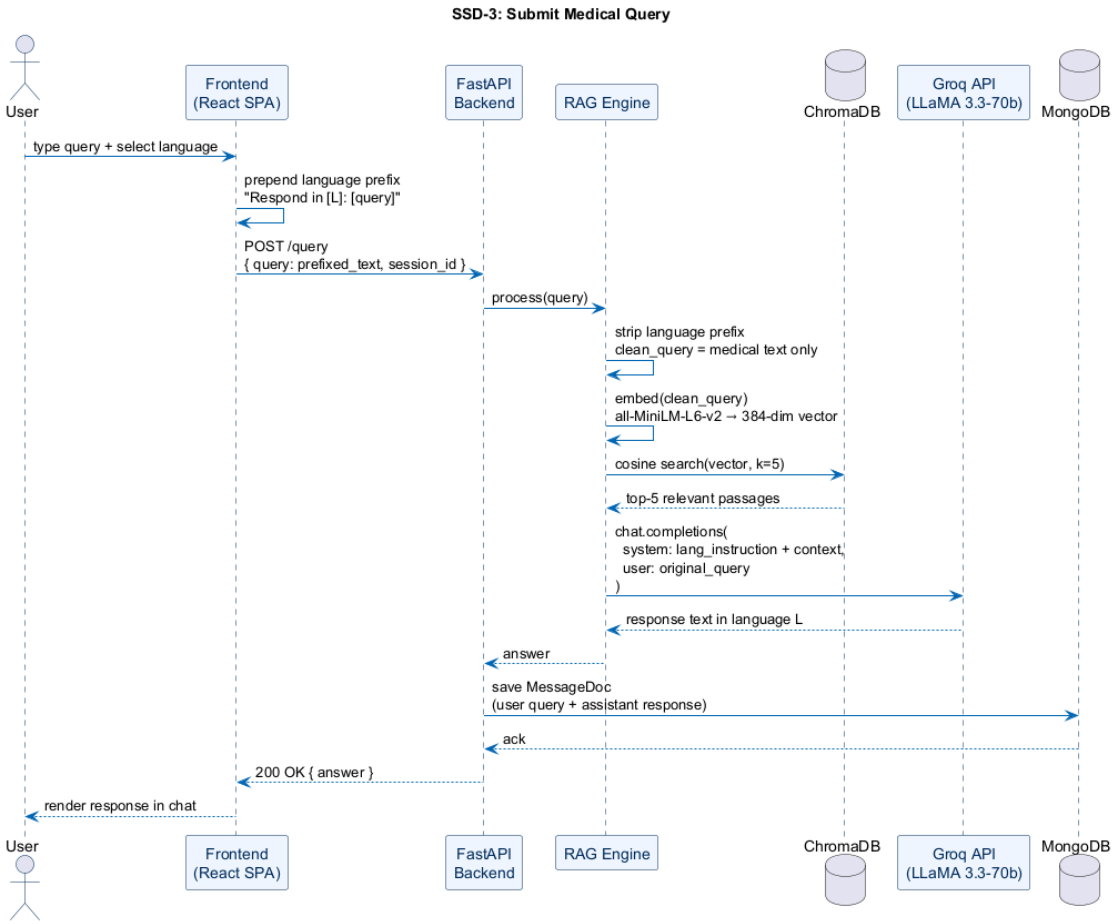



**Figure 2.5:** SSD-3: Submit Medical Query

### 2.8.4 SSD-4: Manage Chat Sessions

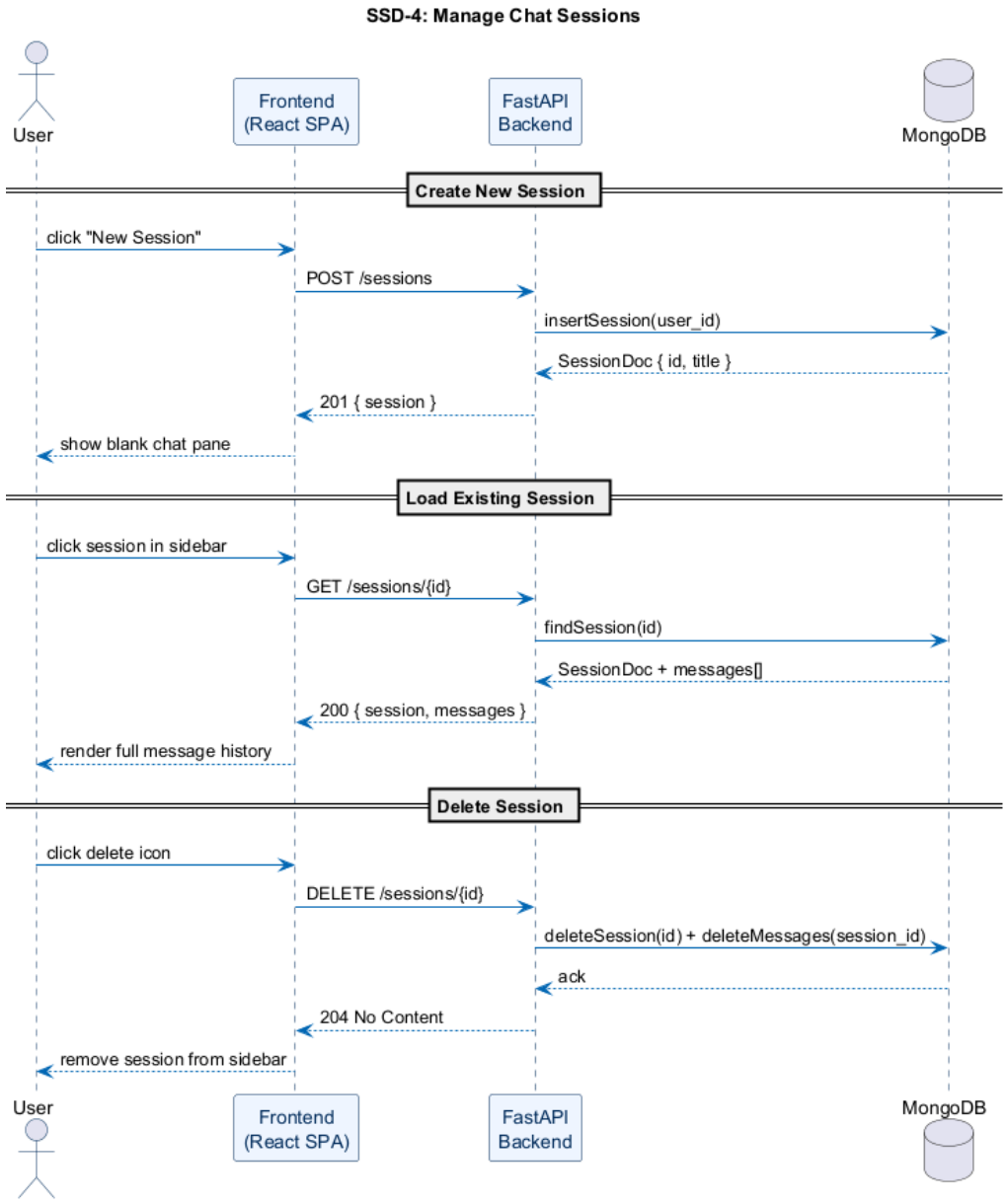



**Figure 2.6:** SSD-4: Manage Chat Sessions (Create / Load / Delete)

### 2.8.5 SSD-5: Use Voice Input

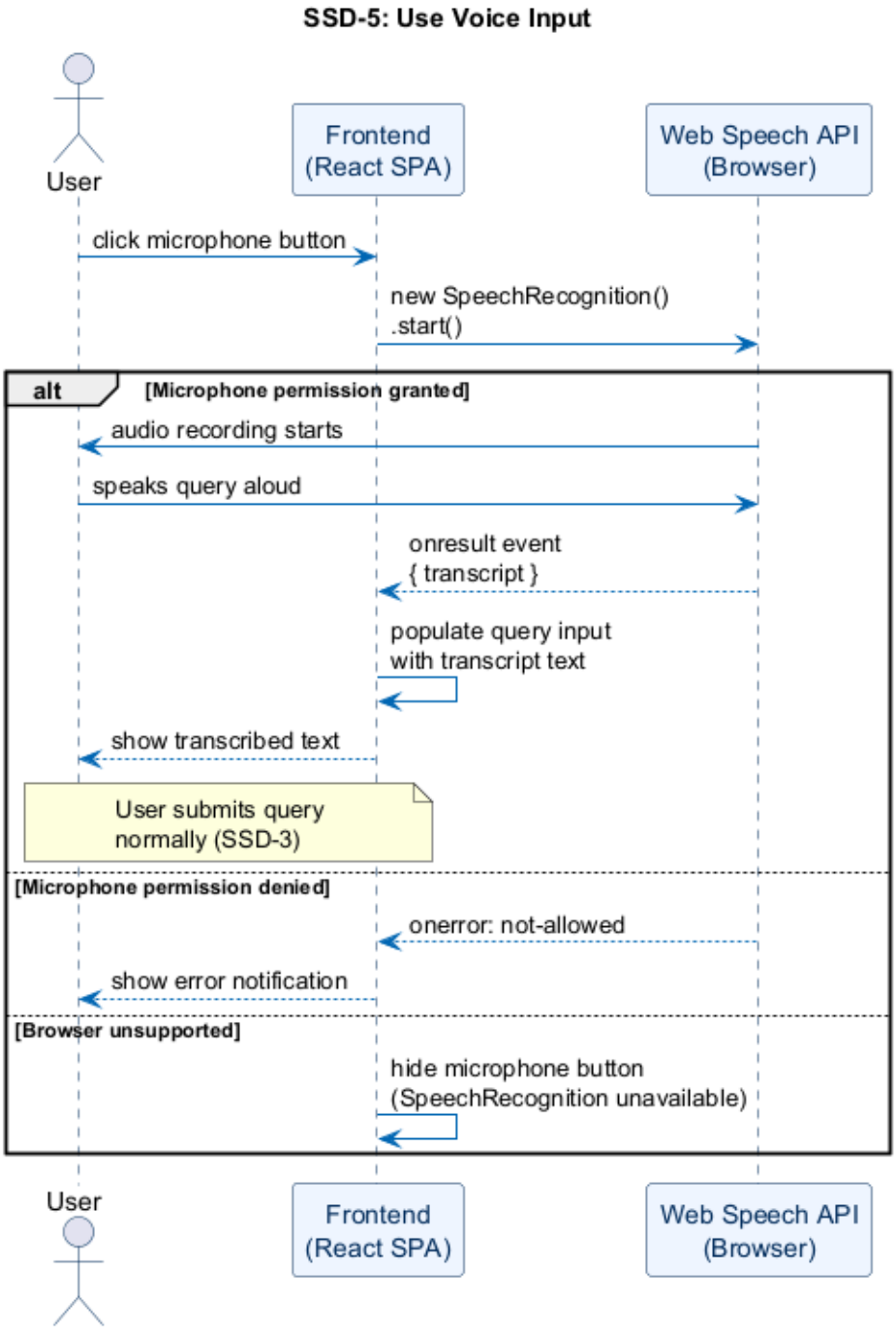



**Figure 2.7:** SSD-5: Use Voice Input

### 2.8.6 SSD-6: Use Voice Output

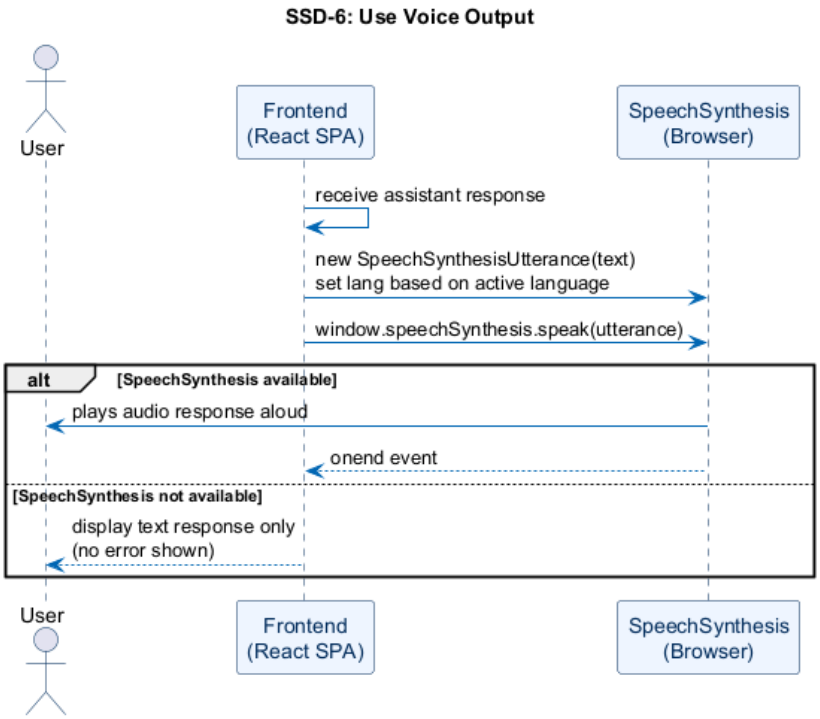



**Figure 2.8:** SSD-6: Use Voice Output

### 2.8.7 SSD-7: Manage Profile

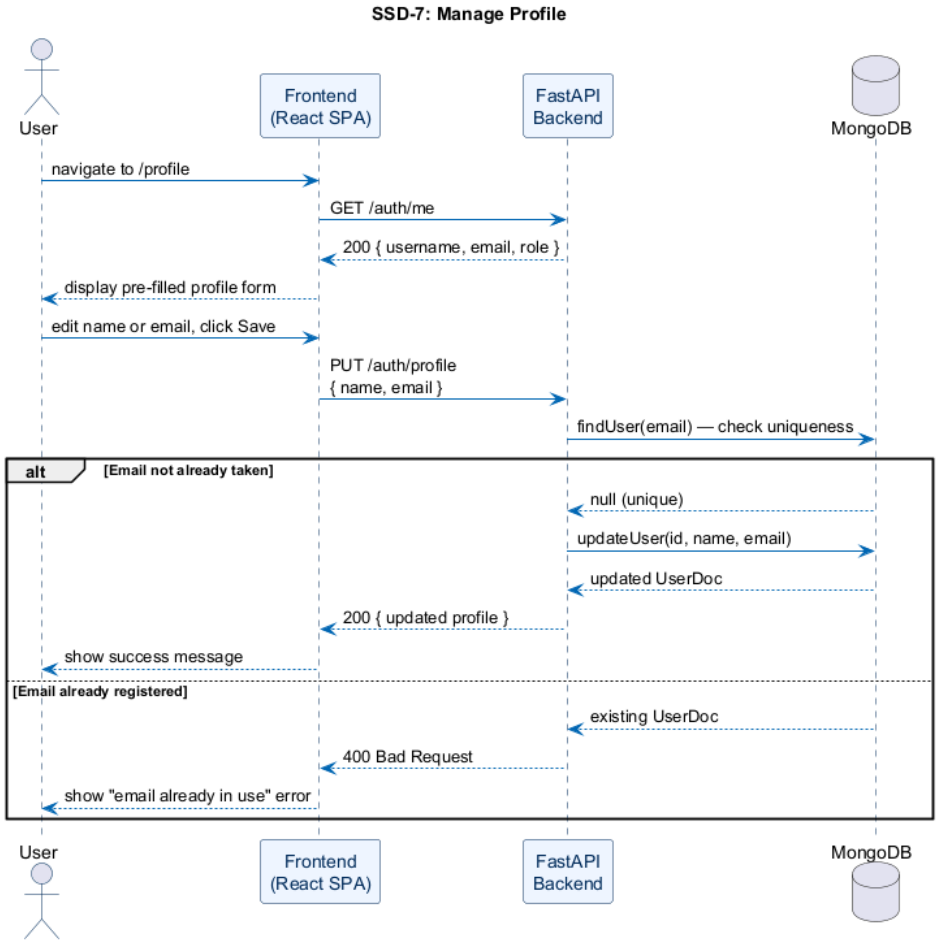



**Figure 2.9:** SSD-7: Manage Profile

### 2.8.8 SSD-8: Reset Password

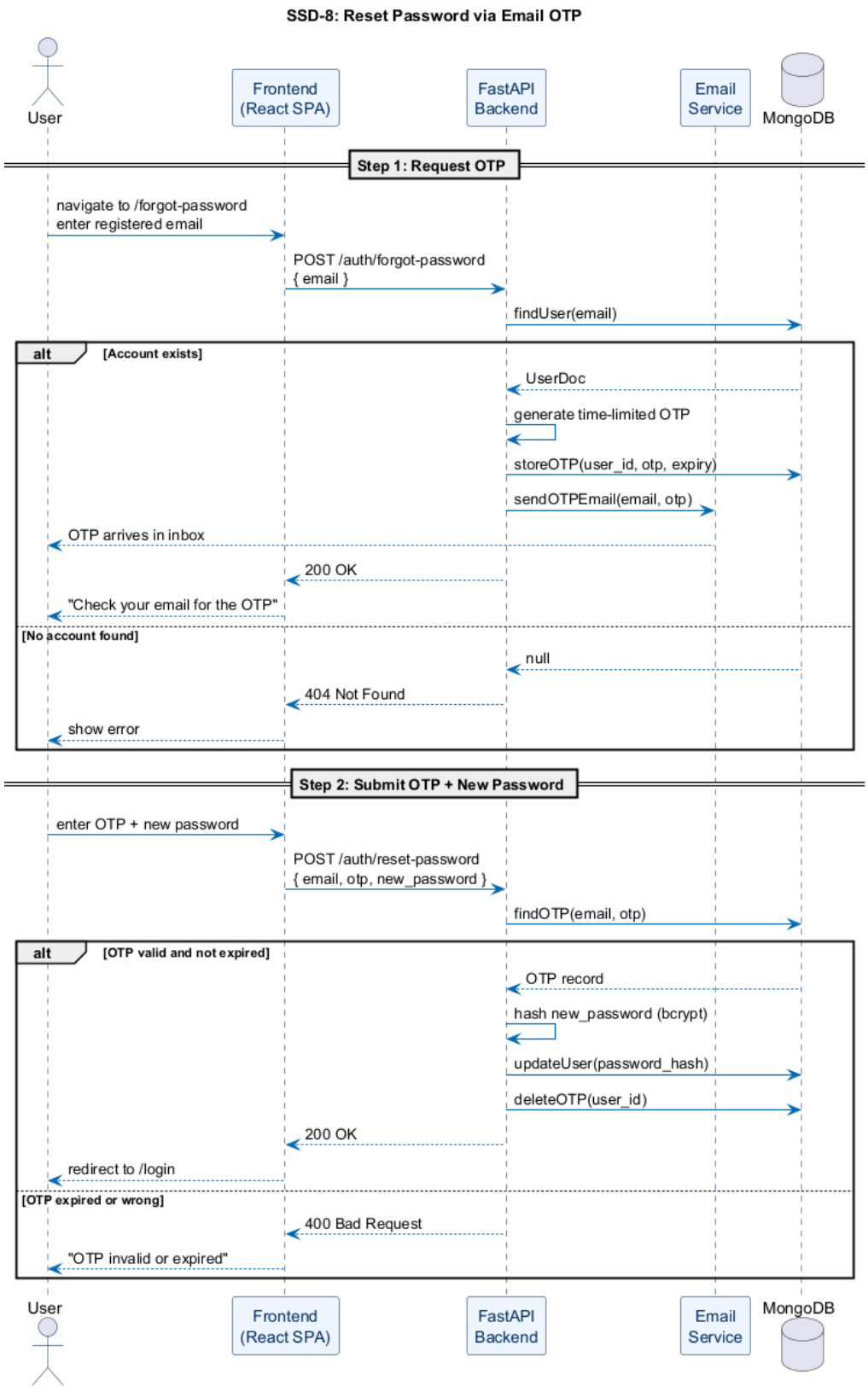



**Figure 2.10:** SSD-8: Reset Password via Email OTP

### 2.8.9 SSD-9: Manage Users (Admin)

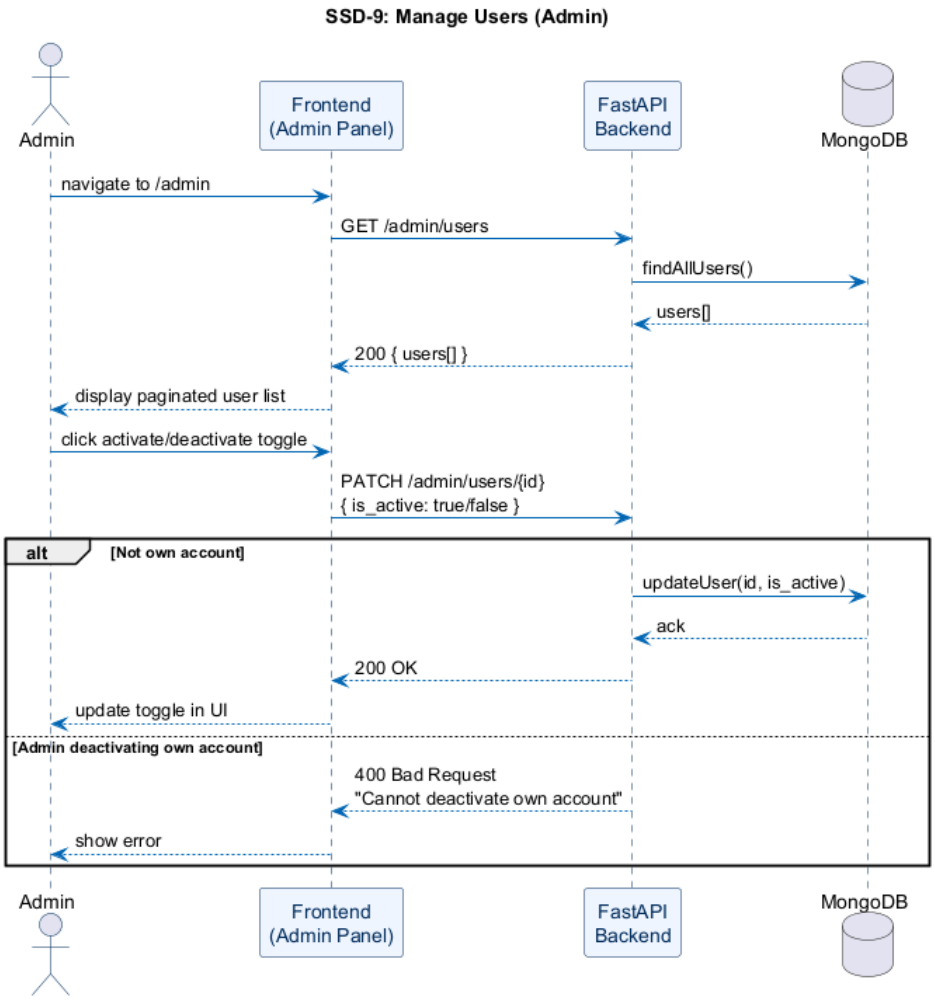



**Figure 2.11:** SSD-9: Manage Users (Admin)

### 2.8.10 SSD-10: Ingest Medical Documents (Admin)

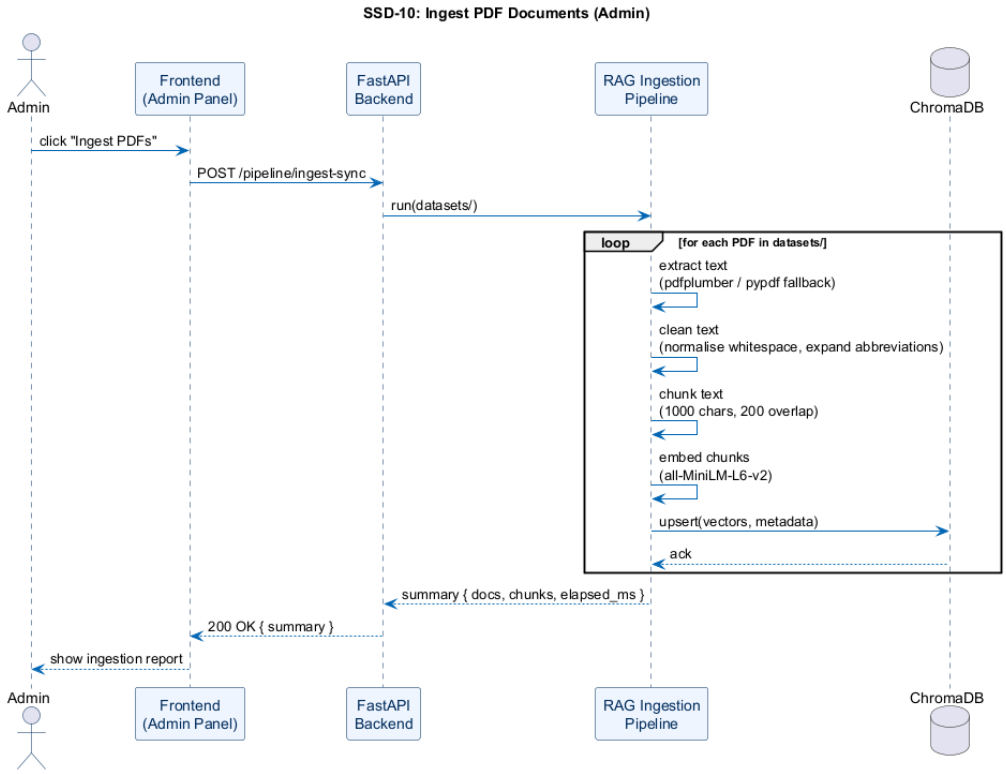



**Figure 2.12:** SSD-10: Ingest Medical Documents (Admin)

## 2.9 Domain Model

The domain model in Figure 2.13 shows the three core persistent entities and how they relate. The structure is deliberately simple. A **User** owns zero or more **Sessions**. Each **Session** contains zero or more **Messages**. When a session is deleted, everything under it goes with it — there are no orphaned messages left behind in the database. All three entities are stored in MongoDB; messages are embedded inside their parent session document rather than in a separate collection.

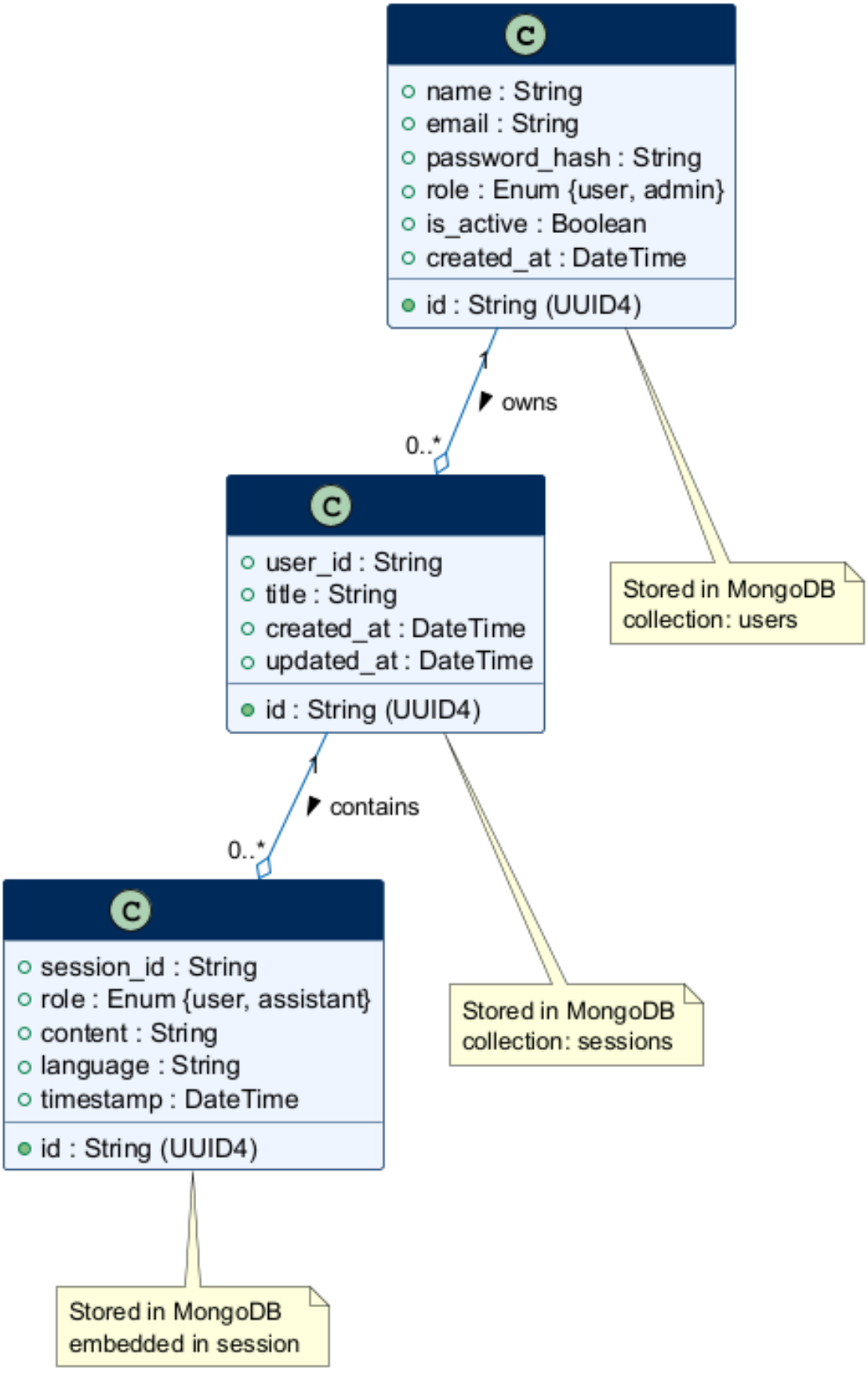



**Figure 2.13:** MediLingo AI — Domain Model

CHAPTER 3

# SOFTWARE DESIGN DESCRIPTION (SDD)

## 3.1 Introduction

### 3.1.1 Design Overview

This Software Design Description (SDD) translates the functional and non-functional requirements from the SRS into concrete structural decisions: how the system is split into layers and components, how those components interact at runtime, and how persistent data is organised. The design choices here are not arbitrary — each one maps back to one or more requirements, and the traceability matrix in Section 3.1.2 makes those mappings explicit. The chapter closes with a class diagram covering the key domain entities and the backend service layer.

MediLingo AI uses a **three-tier client–server architecture**: a React single-page application running in the browser, a stateless FastAPI application server, and a dual persistence layer made up of MongoDB for user and session data and ChromaDB for the indexed medical knowledge base. The three tiers communicate over HTTPS with JSON payloads; refresh tokens travel as HTTP-only cookies, keeping them away from JavaScript running in the page. No business logic lives in the frontend — the SPA is purely responsible for rendering state it receives from the API.

### 3.1.2 Requirements Traceability Matrix

Table 3.1 maps each functional requirement from the SRS to the design component responsible for satisfying it, along with the section of this chapter where that component is described in detail. Any requirement missing a matching row would be a gap in the design.

**Table 3.1:** Requirements Traceability Matrix

| Req. ID | Requirement Summary | Design Component | Section |
|---|---|---|---|
| **FR-01** | User Registration | Auth Router, UserDoc Model | 3.3.2 |
| **FR-02** | User Login | Auth Router, JWT Service | 3.3.2 |
| **FR-03** | Session Persistence (JWT) | Auth Router, apiClient | 3.3.2 |
| **FR-04** | Logout | Auth Router | 3.3.2 |
| **FR-05** | Submit Medical Query | Query Router, RAG Engine | 3.3.4 |
| **FR-06** | Language Selection | Frontend Language Store, RAG Engine Prefix Stripping | 3.3.1 |
| **FR-07** | Multilingual Response | RAG Engine, LLM System Prompt | 3.3.5 |
| **FR-08** | Retrieve Context Chunks | RAG Engine, Vector Store | 3.3.5 |
| **FR-09** | Create Chat Session | Sessions Router, SessionDoc | 3.3.3 |
| **FR-10** | Persist Chat History | Sessions Router, MessageDoc | 3.3.3 |
| **FR-11** | Load Previous Sessions | Sessions Router | 3.3.3 |
| **FR-12** | Delete Session | Sessions Router | 3.3.3 |

*(Table 3.1 continued)*

| Req. ID | Requirement Summary | Design Component | Section |
|---|---|---|---|
| **FR-13** | Voice Input | useVoiceInput Hook (Web Speech API) | 3.3.1 |
| **FR-14** | Voice Output | useVoiceOutput Hook (Web Speech API) | 3.3.1 |
| **FR-15** | Admin Login | Auth Router (role check) | 3.3.2 |
| **FR-16** | PDF Ingestion (Admin) | Pipeline Router, RAG Ingestion Pipeline | 3.3.4 |

## 3.2 System Architectural Design

### 3.2.1 Chosen System Architecture

MediLingo AI uses a **four-tier layered architecture**:

- **Presentation Tier** — React 19 SPA compiled with Vite, served as static files. Talks to the application tier through REST only; no business logic lives here. UI state is managed by Zustand stores and voice I/O goes through the browser's Web Speech API.
- **Application Tier** — FastAPI server. Validates requests, enforces authentication, coordinates calls to the AI/ML and persistence tiers, and returns structured JSON. Stateless by design, which means scaling horizontally is straightforward in principle.
- **AI / ML Processing Tier** — The RAG engine, initialised once at server startup and held as a singleton. Handles embedding generation (sentence-transformers), vector similarity search (ChromaDB), language-prefix stripping, prompt assembly, and LLM invocation (Groq API, LLaMA 3.3-70b). This is the most computationally intensive part of the stack.

- **Persistence Tier** — Two stores: MongoDB (Motor async driver) for user accounts, sessions and messages; ChromaDB (local persistence) for the encoded medical knowledge base. Both are initialised once at startup via the FastAPI lifespan handler.

Figure 3.1 presents the system architectural diagram.

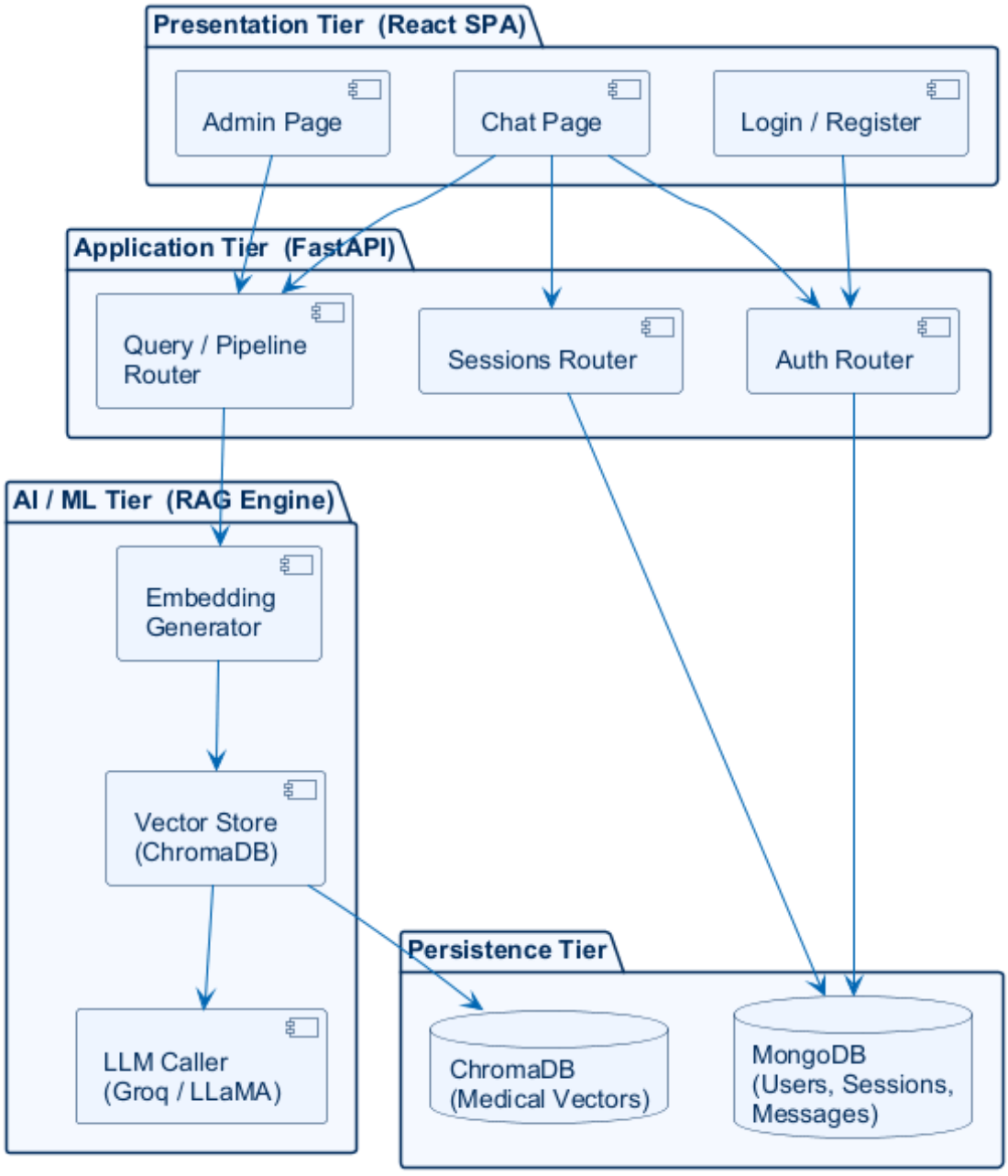



**Figure 3.1:** MediLingo AI — Three-Tier System Architecture

### 3.2.2 Discussion of Alternative Designs

#### *Monolithic vs. Microservices*

A microservices architecture was evaluated — separate deployable units for auth, chat, ingestion and retrieval. It was rejected. The overhead of inter-service communication, distributed tracing, and individual deployment pipelines is real, and the benefit at this scale is not. A single developer building an academic prototype does not need Kubernetes. The layered monolith achieves the same separation of concerns through module boundaries rather than process boundaries, which is much easier to develop, debug and test.

#### *Server-Side Rendering vs. SPA*

Next.js with server-side rendering was considered as an alternative to the Vite SPA. SSR improves initial page load and SEO, which matters for public-facing content. MediLingo AI's chat interface is inherently interactive and session-gated — there is no content for search engines to index, no cold visitors landing on a page. The Vite build was preferred: faster development iteration, simpler deployment as static files, and cleaner integration with Zustand for keeping access tokens in memory rather than exposing them through server-rendered HTML.

#### *Relational vs. Document Store*

PostgreSQL with pgvector was a tempting option — one database handling both relational user data and vector search. ChromaDB plus MongoDB was chosen instead. ChromaDB's purpose-built HNSW index and embedding API make the retrieval path significantly simpler, and MongoDB's flexible schema means the message and session structures can evolve without writing and running migrations every time something changes. For an academic project where the schema is still being iterated on, that flexibility mattered.

### 3.2.3 System Interface Description

- **User Interface** — Browser-based React SPA. Input: text box, microphone button, language selector. Output: streamed chat bubbles, text-to-speech playback.
- **API Interface** — REST over HTTP/HTTPS. Base URL `/api/v1`. Authentication via `Authorization: Bearer <access_token>` header. Refresh via HTTP-only

`refresh_token` cookie sent automatically by the browser.

- **LLM Interface** — Groq REST API. The RAG engine sends a `chat.completions` request containing a system prompt (with language instruction and retrieved context) and the user message. Model: `llama-3.3-70b-versatile`.
- **Database Interface** — Motor (async PyMongo) for MongoDB; ChromaDB Python client for vector operations. Both connections are initialised once at server startup via the FastAPI lifespan handler.

## 3.3 Detailed Description of Components

### 3.3.1 Frontend (React SPA)

The frontend is a React 19 / TypeScript application built with Vite. It is structured around pages, Zustand stores, and custom hooks that keep side-effectful code away from the rendering layer. Most components are stateless; they display data received through props or from the stores, and they call hook functions to trigger any side effects.

- **router.tsx** — All client-side routes, defined with React Router v6. Protected routes check the auth store and redirect to `/login` when no valid access token is present.
- **App.tsx** — The root component. On mount it silently calls `/auth/refresh`; if the refresh cookie is still valid the access token is restored and the user lands directly on the chat page without re-entering credentials. This is how the "stay logged in" experience works.
- **useAuthStore (Zustand)** — Holds `user` and `accessToken` in memory only. Tokens are never written to `localStorage` or `sessionStorage` — they disappear when the tab closes.
- **apiClient.ts** — All API calls go through here. It wraps `fetch` with a configurable timeout and handles 401 responses automatically: calls `/auth/refresh`, updates the store, and retries the original request once. If the retry also fails, it clears auth state and redirects to `/login`.

- **useVoiceInput / useVoiceOutput** — Thin wrappers around `SpeechRecognition` and `SpeechSynthesis`. The output hook picks a locale-appropriate voice based on the active language setting.

### 3.3.2 Authentication Router

`routers/auth.py` exposes five endpoints under `/auth`:

- **POST /register** — Validates uniqueness of both username and email, hashes the password with bcrypt via `passlib`, stores a `UserDoc` in MongoDB, and returns an access/refresh token pair. A duplicate email gets HTTP 400.
- **POST /login** — Verifies credentials against the stored bcrypt hash and issues a new token pair. The refresh token goes into an HTTP-only, Secure, SameSite=Lax cookie with a 7-day TTL.
- **POST /refresh** — Reads the refresh cookie, verifies the JWT signature and expiry, then issues a new 15-minute access token. The remaining TTL on the refresh token is preserved, not reset.
- **POST /logout** — Clears the refresh cookie. Nothing else needs to happen server-side because access tokens are not persisted anywhere.
- **GET /me** — Returns the current user's profile using the access token from the Authorization header.

All tokens are signed HS256 JWTs. The `SECRET_KEY` used for signing is loaded from the environment at startup and is never present in the source code.

### 3.3.3 Sessions Router

`routers/sessions.py` manages chat sessions and their messages under `/sessions`. Every endpoint here requires a valid access token.

- **POST /sessions** — Creates a new `SessionDoc` in MongoDB with a UUID, the authenticated user's ID, a display title and an empty message list.
- **GET /sessions** — Returns the user's sessions, sorted by last-updated descending.

This is what populates the sidebar.

- **GET /sessions/{id}** — Returns a single session with its full message history. Called when a user clicks on a session to resume it.
- **POST /sessions/{id}/messages** — Appends a `MessageDoc` (role, content, timestamp, language) to the session. Called by the query router after each exchange, not directly by the user.
- **DELETE /sessions/{id}** — Deletes the session and all its embedded messages.

### 3.3.4 Query and Pipeline Routers

`routers/query.py` handles all AI-related endpoints:

- **POST /query** — The primary chat endpoint. Accepts `query` (the full language-prefixed string), strips the prefix, embeds the clean query, retrieves the top-$k$ context chunks from ChromaDB, assembles the Groq prompt, and streams the LLM response back to the caller.
- **POST /retrieve** — Returns only the raw retrieved context chunks, without LLM generation. Used for debugging and evaluation.
- **GET /config** — Returns current RAG configuration (chunk size, overlap, top-k, model name).
- **GET /info/{collection}** — Returns metadata about a ChromaDB collection (document count, embedding dimension).

`routers/query.py` also contains pipeline management endpoints under `/pipeline`:

- **POST /pipeline/ingest-sync** — Synchronously ingests all PDF files found in `backend/datasets/`. Admin-only. Returns a summary of documents indexed, chunks created, and time elapsed.
- **POST /pipeline/ingest** — Same as above but runs as a FastAPI `BackgroundTask`, returning immediately with a task-accepted response.

### 3.3.5 RAG Engine

The RAG engine (`rag/engine.py`) is the core AI component of the system. It is initialised once at server startup through `services/rag_service.py` and held as a module-level singleton. The processing pipeline for each incoming query has five steps:

1. **Prefix Stripping** — The language instruction prefix (e.g., *"Respond in Urdu: "*) is detected and stripped before anything else happens. This step is critical for retrieval quality: the clean medical question is what gets embedded and searched, not the prefixed string.
2. **Embedding** — The clean query is passed to `EmbeddingGenerator.encode()`, which runs `all-MiniLM-L6-v2` via `sentence-transformers` to produce a 384-dimensional dense vector.
3. **Vector Retrieval** — The 384-dim query vector is submitted to `ChromaDBVectorStore.query()`, which returns the top-$k$ document chunks by cosine similarity.
4. **Prompt Assembly** — A structured Groq chat-completions prompt is built: the system message contains the language instruction, a medical-domain persona and the retrieved context passages; the user message contains the original prefixed query.
5. **LLM Invocation** — The assembled prompt is sent to the Groq API (`llama-3.3-70b-versatile`). The response text is returned to the calling router, which saves it to MongoDB and returns it to the frontend.

### 3.3.6 Ingestion Pipeline

`rag/pipeline.py` contains `RAGIngestionPipeline`, which handles the one-time (or on-demand) process of building the medical knowledge base:

1. **PDF Loading** — `rag/loader.py` uses `pdfplumber` as the primary extractor, falling back to `pypdf` for files that pdfplumber cannot handle cleanly. Every PDF in `backend/datasets/` is processed.
2. **Text Cleaning** — `rag/preprocessor.py` normalises whitespace, expands common medical abbreviations, strips headers and footers, and removes non-printable

characters. The goal is clean, consistent text going into the embedder.

3. **Semantic Chunking** — `rag/chunker.py` splits cleaned text into fixed-size chunks of 1000 characters with 200-character overlap. The overlap ensures sentences split across a chunk boundary still appear complete in at least one chunk. Short chunks below a minimum length are discarded.

4. **Embedding and Indexing** — Each chunk is embedded and upserted into the ChromaDB collection with source filename and chunk index as metadata. Upsert rather than insert means re-running the pipeline on an already-indexed document does not create duplicates.

### 3.3.7 Data Models

Three MongoDB document classes are defined in `models/`:

- **UserDoc** — `id` (UUID4 str), `username`, `email`, `hashed_password`, `role` (*user* or *admin*), `created_at`. Unique indexes on `username` and `email` are created at startup to enforce deduplication at the database level.

- **SessionDoc** — `id` (UUID4 str), `user_id`, `title`, `created_at`, `updated_at`, and `messages` — an embedded array of `MessageDoc`. The index on `user_id` makes the "fetch my sessions" query fast regardless of collection size.

- **MessageDoc** — `id` (UUID4 str), `role` (*user* or *assistant*), `content`, `language`, `timestamp`. Stored embedded inside `SessionDoc` rather than in a separate collection.

All three models expose `to_mongo()` and `from_mongo()` helpers. UUID4 string IDs are used throughout to decouple application identity from MongoDB's internal `_id` field — application code never touches `_id` directly.

## 3.4 Architectural Diagram

Figure 3.1 in Section 3.2.1 shows the four-tier layered architecture. Figure 3.2 below zooms in on the AI/ML tier and traces the full query processing flow from the moment

a prefixed query arrives at the RAG engine to the moment a response is returned to the router.

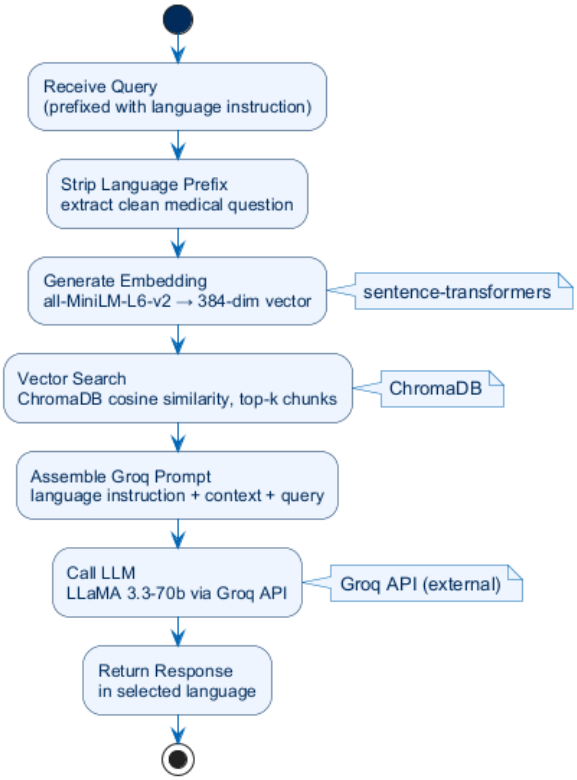



**Figure 3.2:** RAG Query Processing Flow

## 3.5 Sequence Diagrams

This section presents sequence diagrams for each of the ten use cases identified in the SRS. All ten diagrams are included as exported PNG sequence diagrams.

### 3.5.1 SSD-1: Register Account

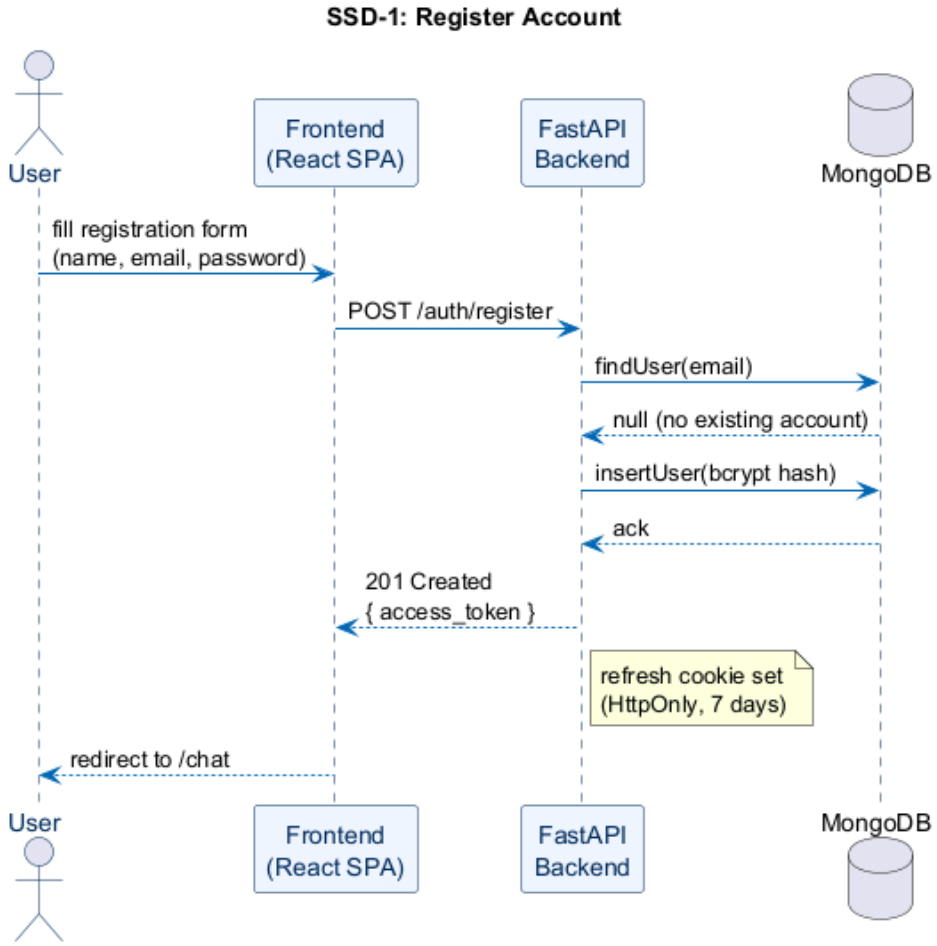



**Figure 3.3:** SSD-1: Register Account

### 3.5.2 SSD-2: Login

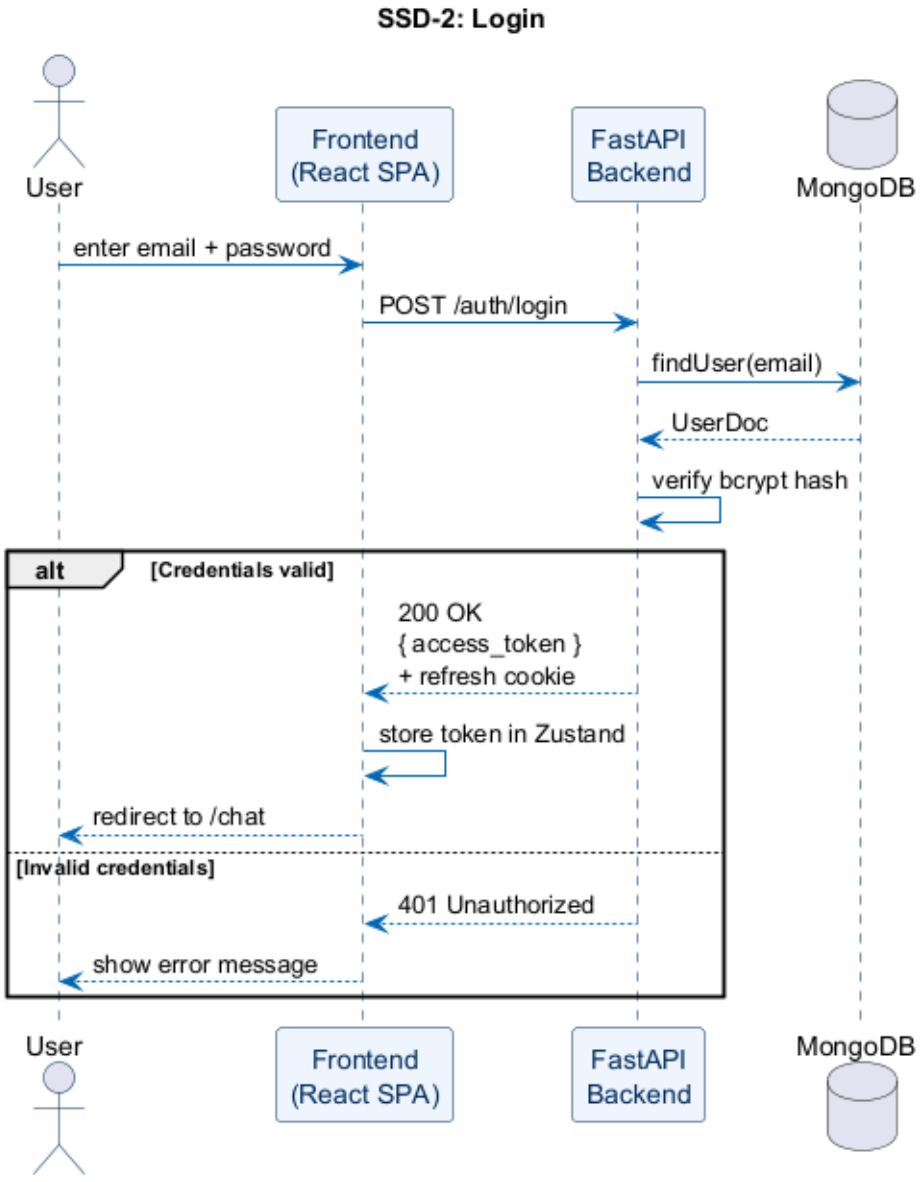



**Figure 3.4:** SSD-2: Login

### 3.5.3 SSD-3: Submit Medical Query

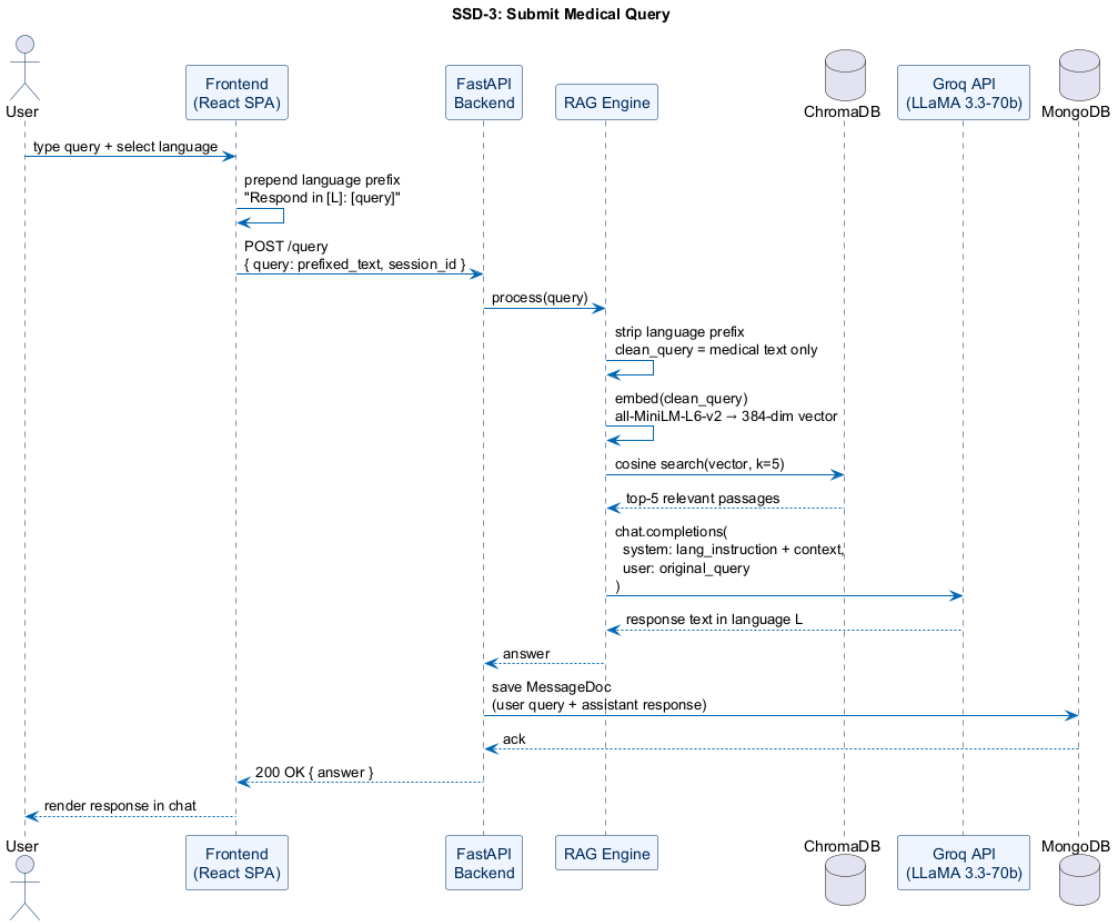



**Figure 3.5:** SSD-3: Submit Medical Query

### 3.5.4 SSD-4: Voice Input

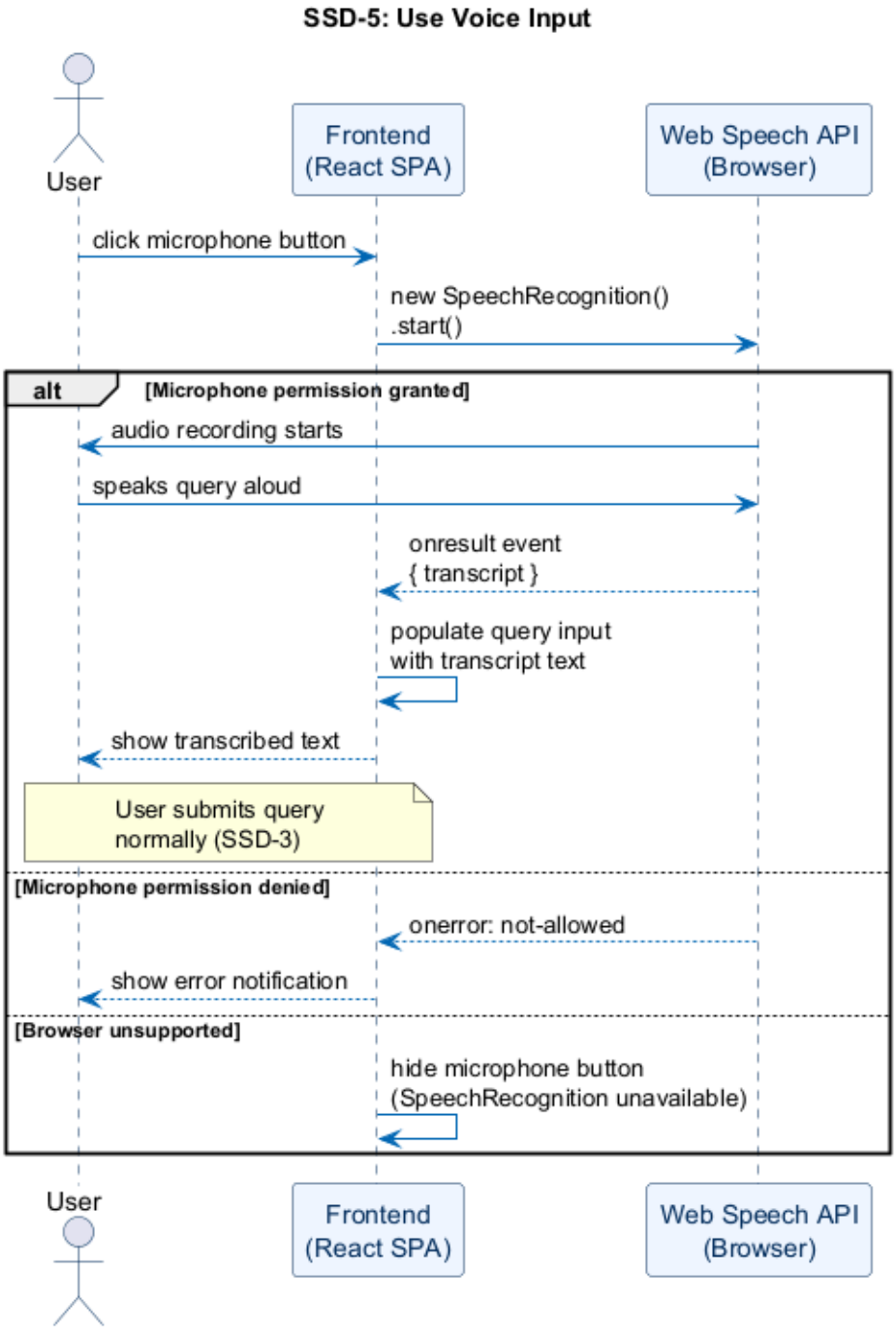



**Figure 3.6:** SSD-4: Voice Input — Speech Recognition Flow

### 3.5.5 SSD-5: Voice Output

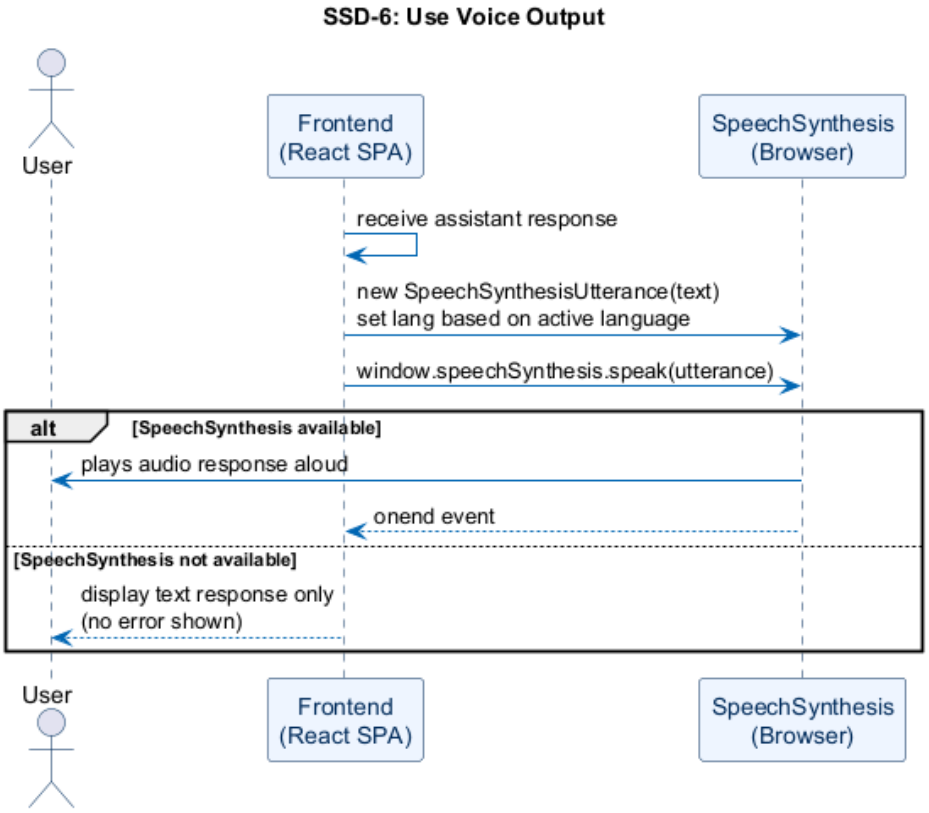



**Figure 3.7:** SSD-5: Voice Output — Text-to-Speech Playback Flow

### 3.5.6 SSD-6: Create New Session

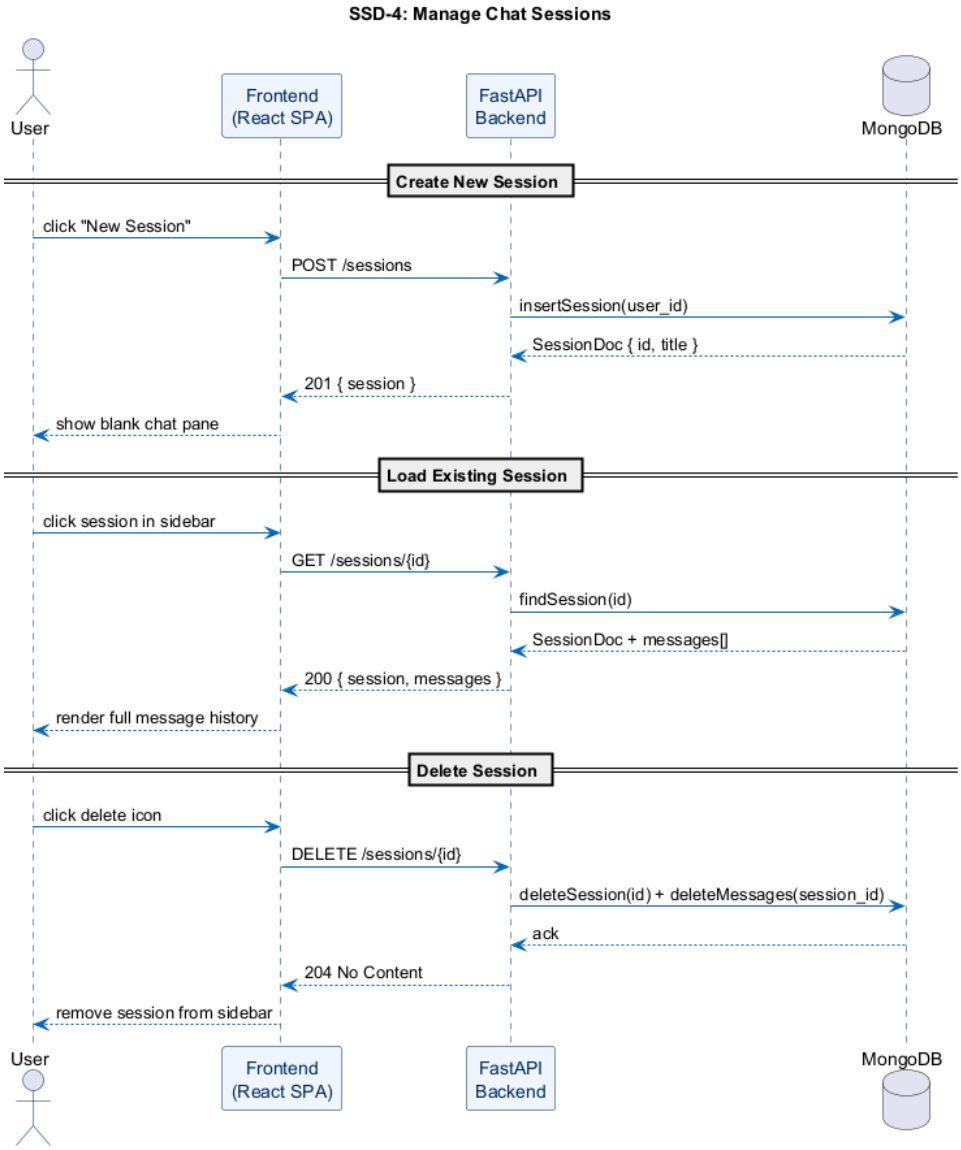



**Figure 3.8:** SSD-6: Create New Session

### 3.5.7 SSD-7: Load Previous Sessions

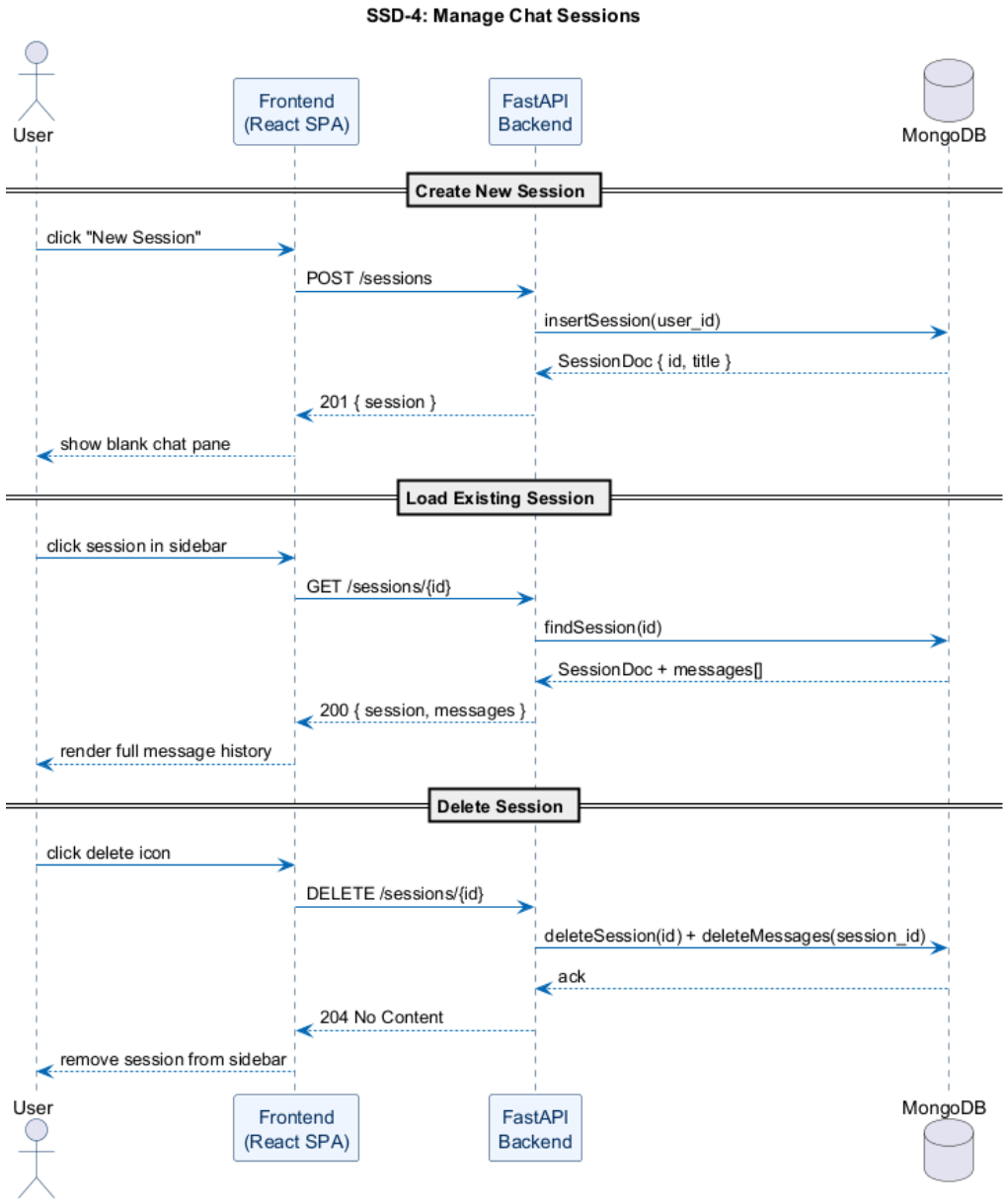



**Figure 3.9:** SSD-7 & SSD-8: Session Management — Create, Load, and Delete

### 3.5.8 SSD-8: Delete Session

The delete session flow is covered in Figure 3.9 above, which shows all three session management operations (create, load, and delete) in a single combined sequence diagram.

### 3.5.9 SSD-9: Admin Login

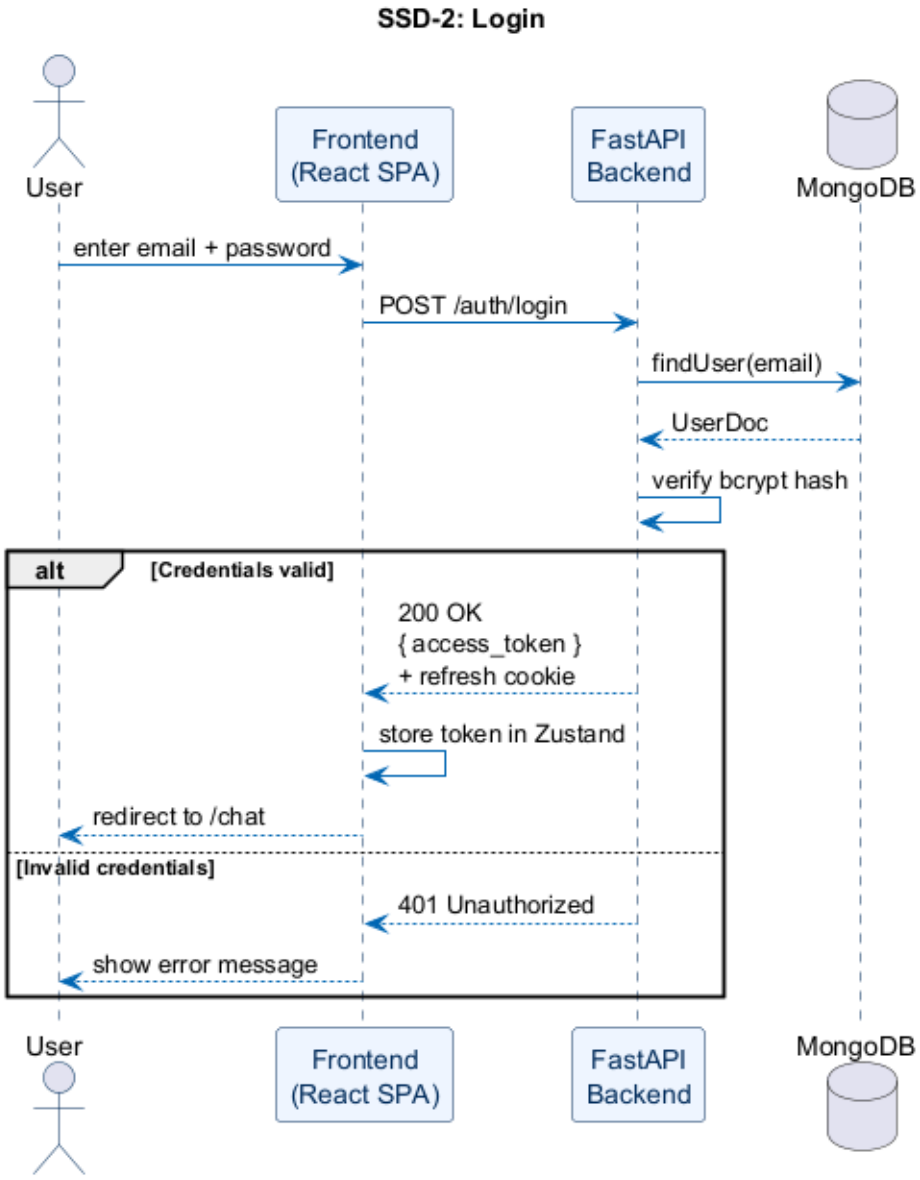



**Figure 3.10:** SSD-9: Admin Login — same auth flow as SSD-2, role claim verified server-side

### 3.5.10 SSD-10: Ingest PDF Documents

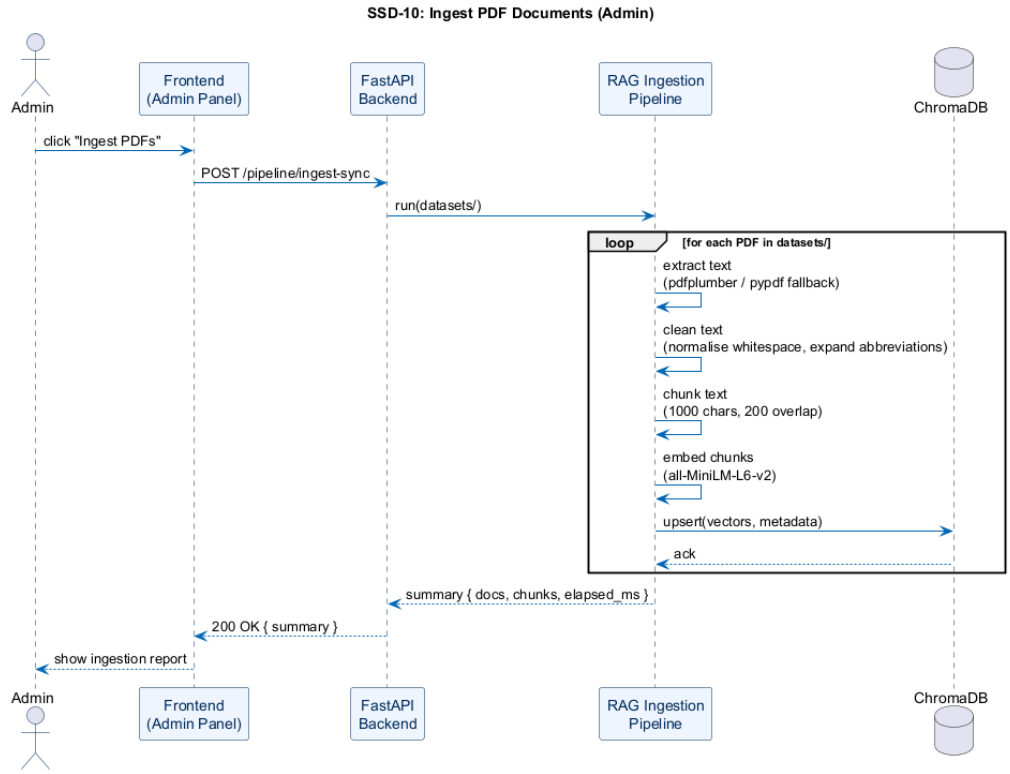



**Figure 3.11:** SSD-10: Ingest PDF Documents

## 3.6 Class Diagram

Figure 3.12 shows the key domain classes, their attributes and their relationships. Frontend Zustand stores are left out — this diagram focuses on the backend data model and the service layer that drives it. The full picture would include the React hooks and stores, but adding them here would make the diagram unreadable without adding much that is not already covered in the frontend component description.

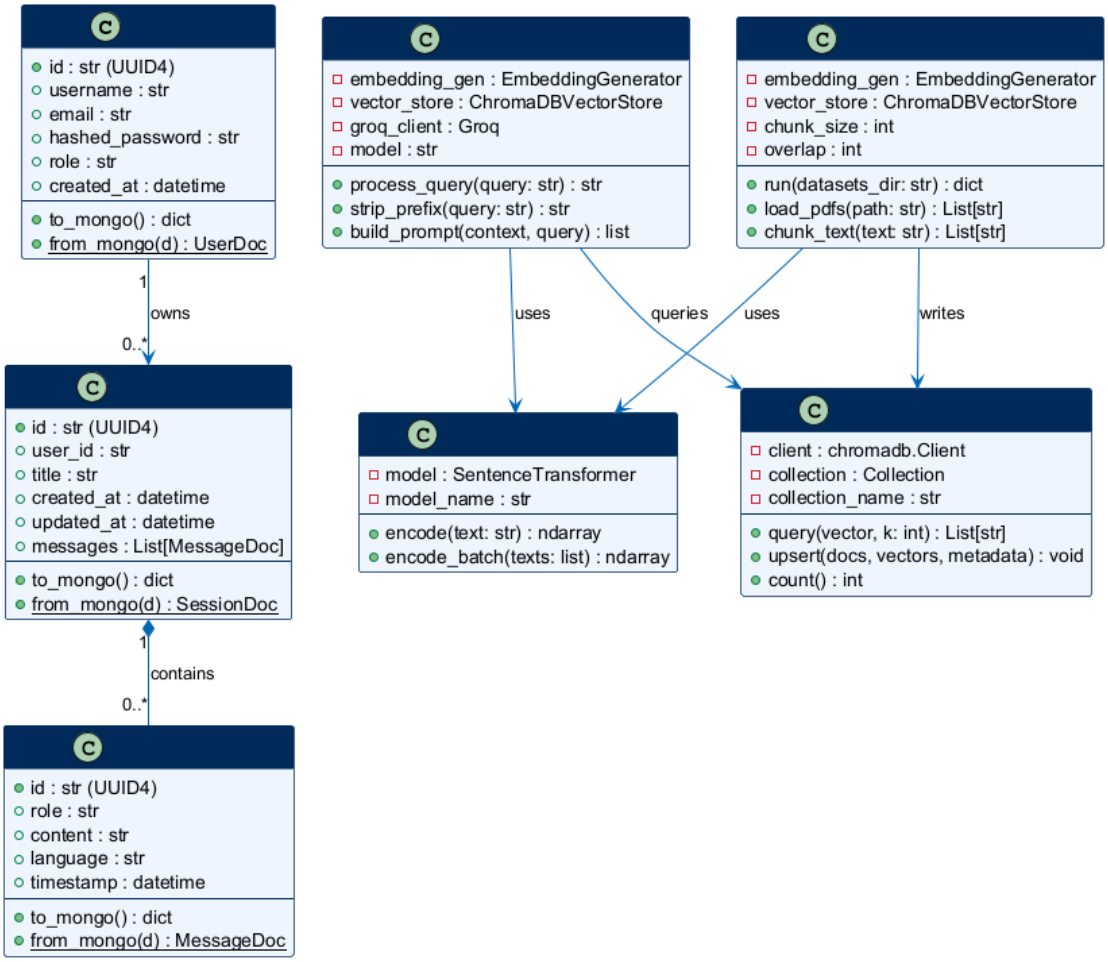



**Figure 3.12:** MediLingo AI — Class Diagram

Chapter 4

# SOFTWARE TEST DOCUMENTATION (STD)

## 4.1 Introduction

### 4.1.1 System Overview

Testing any software system is, at its core, about building confidence. Not certainty — no test suite can guarantee that — but enough confidence that the system behaves the way it was designed to behave under the conditions it will actually face. For MediLingo AI, that means verifying that a user can register, log in, ask a medical question in Urdu or English, get a sensible answer back, and have that conversation saved for later. It also means making sure the admin can upload new PDF documents and that the knowledge base updates correctly.

The system under test is a full-stack web application: a React frontend talking to a FastAPI backend, which in turn calls the Groq API and queries ChromaDB. MongoDB stores everything that needs to persist. Because there are several moving parts here, testing was done at multiple levels — individual API endpoints, the authentication flow end to end, and basic UI interactions through the browser.

### 4.1.2 Test Approach

The primary testing method was **black-box functional testing**. Each test case was written from the perspective of what a user or admin would actually do, not from knowledge of the internal implementation. The question being asked in every test is simple: given this input, does the system produce the right output?

Two categories of scenarios were covered for each feature: the *happy path* (everything

goes as expected) and at least one *negative path* (what happens when something is wrong — bad credentials, missing fields, duplicate entries, and so on). Systems that only work when everything is correct are not production-ready, and that is as true for an academic project as for anything else.

API endpoints were tested directly using Postman, which made it easy to craft specific request bodies and inspect raw responses without the frontend getting in the way. UI flows were verified manually in Google Chrome. The RAG pipeline was tested by submitting known medical queries and checking that the response was grounded in the indexed documents rather than being a hallucination.

Worth noting: no automated test framework such as Pytest or Vitest was used for these tests. The test cases documented here represent manual test runs. The pass/fail results reflect observed system behaviour during the testing phase.

## 4.2 Test Plan

### 4.2.1 Features to be Tested

The following features were included in the test scope:

- User registration and login, including duplicate account prevention and wrong password handling
- JWT refresh flow — specifically, whether an expired access token is transparently renewed using the refresh cookie
- Logout and session clearing
- Submitting medical queries in English, Urdu, and Roman Urdu
- Language prefix stripping — verifying that the embedding pipeline receives the clean query, not the prefixed one
- Session creation, retrieval, and deletion
- Chat history persistence across page reloads

- Voice input (speech to text) and voice output (text to speech)
- Admin login with role enforcement
- PDF ingestion via the admin panel and subsequent retrieval from ChromaDB

### 4.2.2 Features Not to be Tested

Some things are deliberately out of scope:

- **Groq API internals** — the LLM is an external service. Testing whether LLaMA 3.3-70b produces medically accurate responses is beyond the scope of this project; what is tested here is whether the system correctly calls the API and returns the response to the user.
- **sentence-transformers model accuracy** — the embedding model is a pre-trained open-source model (`all-MiniLM-L6-v2`). Its retrieval quality on general benchmarks is well documented. What is tested here is whether the integration works, not the model itself.
- **Cross-browser compatibility** — testing was done in Chrome only. Firefox and Safari compatibility was not verified.
- **Load and stress testing** — the system was not tested under high concurrent user load. This is a single-developer academic prototype, not a production service.
- **MongoDB replication or failover** — a single local MongoDB instance was used throughout. Cluster behaviour is outside scope.

### 4.2.3 Testing Tools and Environment

**Table 4.1:** Testing Environment

| Component | Details |
|---|---|
| Operating System | Windows 11 Home (64-bit) |
| Browser | Google Chrome (latest stable) |
| API Testing Tool | Postman |
| Backend Runtime | Python 3.13, FastAPI with Uvicorn |
| Frontend Runtime | Node.js 20, Vite dev server (port 5173) |
| Database | MongoDB 7.x (local instance, port 27017) |
| Vector Store | ChromaDB (local persistence, `backend/vectorstore/`) |
| LLM API | Groq Cloud (LLaMA 3.3-70b-versatile) |
| Test PDF | Sample medical reference document placed in `backend/datasets/` |

## 4.3 Test Cases

Each use case from the SRS gets at least one positive test and one negative test below. Test IDs follow the format TC-*UC*-*N*, where *UC* is the use case number and *N* distinguishes multiple tests within the same use case. Results reflect observed system behaviour during the testing phase, not theoretical expectations.

### 4.3.1 UC-1: Register Account

**Table 4.2:** TC-01-1: Successful Registration

| Field | Details |
|---|---|
| Test Case ID | TC-01-1 |
| Use Case | UC-1: Register Account |
| Objective | Verify that a new user can create an account with valid details |
| Preconditions | Backend running; no existing account with the test email |
| Test Input | username: `testuser01`, email: `testuser01@mail.com`, password: `Test@1234` |
| Expected Output | HTTP 201; access token returned in response body; refresh cookie set; user redirected to chat page |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.3:** TC-01-2: Registration with Duplicate Email

| Field | Details |
|---|---|
| Test Case ID | TC-01-2 |
| Use Case | UC-1: Register Account |
| Objective | Verify that registering with an already-used email is rejected |
| Preconditions | Account with `testuser01@mail.com` already exists |
| Test Input | Same email as TC-01-1 with a different username |
| Expected Output | HTTP 400; error message indicating email already in use |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.2 UC-2: Login

**Table 4.4:** TC-02-1: Successful Login

| Field | Details |
|---|---|
| Test Case ID | TC-02-1 |
| Use Case | UC-2: Login |
| Objective | Verify that a registered user can log in with correct credentials |
| Preconditions | Account from TC-01-1 exists |
| Test Input | username: `testuser01`, password: `Test@1234` |
| Expected Output | HTTP 200; access token in body; HttpOnly refresh cookie set; user lands on chat page |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.5:** TC-02-2: Login with Wrong Password

| Field | Details |
|---|---|
| Test Case ID | TC-02-2 |
| Use Case | UC-2: Login |
| Objective | Verify that an incorrect password is rejected |
| Preconditions | Account from TC-01-1 exists |
| Test Input | username: `testuser01`, password: `wrongpass` |
| Expected Output | HTTP 401; error message: “Invalid credentials” |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.3 UC-3: Submit Medical Query

**Table 4.6:** TC-03-1: English Query

| Field | Details |
|---|---|
| Test Case ID | TC-03-1 |
| Use Case | UC-3: Submit Medical Query |
| Objective | Verify that an English medical query returns a relevant, grounded response |
| Preconditions | User logged in; at least one PDF ingested into ChromaDB |
| Test Input | Language: English; query: “What are the symptoms of diabetes?” |
| Expected Output | HTTP 200; response text in English describing diabetes symptoms; content references retrieved context |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.7:** TC-03-2: Urdu Query

| Field | Details |
|---|---|
| Test Case ID | TC-03-2 |
| Use Case | UC-3: Submit Medical Query |
| Objective | Verify that an Urdu query returns a response written in Urdu |
| Preconditions | User logged in; language selector set to Urdu |
| Test Input | Language: Urdu; query: *(What are the symptoms of diabetes? — written in Urdu script)* |
| Expected Output | Response text in Urdu script; semantically correct answer |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.8:** TC-03-3: Roman Urdu Query

| Field | Details |
|---|---|
| Test Case ID | TC-03-3 |
| Use Case | UC-3: Submit Medical Query |
| Objective | Verify that a Roman Urdu query returns a response in Roman Urdu |
| Preconditions | User logged in; language selector set to Roman Urdu |
| Test Input | Language: Roman Urdu; query: “Diabetes ki alamaat kya hain?” |
| Expected Output | Response written in Roman Urdu; content correct |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.9:** TC-03-4: Query Without Ingested Documents

| Field | Details |
|---|---|
| Test Case ID | TC-03-4 |
| Use Case | UC-3: Submit Medical Query |
| Objective | Verify graceful handling when the vector store is empty |
| Preconditions | ChromaDB collection empty (no ingestion run yet) |
| Test Input | Any valid medical query |
| Expected Output | System responds with a message indicating no context is available; does not crash |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.4 UC-4: Voice Input

**Table 4.10:** TC-04-1: Voice Input in English

| Field | Details |
|---|---|
| Test Case ID | TC-04-1 |
| Use Case | UC-4: Voice Input |
| Objective | Verify that speech is correctly transcribed and placed in the query box |
| Preconditions | Chrome browser; microphone permission granted; language set to English |
| Test Input | Spoken: “What is hypertension?” |
| Expected Output | Query box populated with transcribed text; query submittable |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.11:** TC-04-2: Microphone Permission Denied

| Field | Details |
|---|---|
| Test Case ID | TC-04-2 |
| Use Case | UC-4: Voice Input |
| Objective | Verify the app handles microphone permission denial gracefully |
| Preconditions | Microphone permission blocked in Chrome settings |
| Test Input | Click microphone button |
| Expected Output | Error notification shown to user; app does not crash |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.5 UC-5: Voice Output

**Table 4.12:** TC-05-1: Text-to-Speech Playback

| Field | Details |
|---|---|
| Test Case ID | TC-05-1 |
| Use Case | UC-5: Voice Output |
| Objective | Verify that the system reads out the assistant response after it arrives |
| Preconditions | User logged in; a query has just been submitted and answered |
| Test Input | Language: English; response received from the system |
| Expected Output | Browser TTS reads the response aloud using an English voice |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.6 UC-6: Create New Session

**Table 4.13:** TC-06-1: Create a New Chat Session

| Field | Details |
|---|---|
| Test Case ID | TC-06-1 |
| Use Case | UC-6: Create New Session |
| Objective | Verify that clicking New Session creates a blank session in the sidebar |
| Preconditions | User is logged in and on the chat page |
| Test Input | Click the “New Session” button |
| Expected Output | New session appears in sidebar; chat area is blank; session ID stored in MongoDB |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.7 UC-7: Load Previous Sessions

**Table 4.14:** TC-07-1: Reload Previous Session After Page Refresh

| Field | Details |
|---|---|
| Test Case ID | TC-07-1 |
| Use Case | UC-7: Load Previous Sessions |
| Objective | Verify that a session with prior messages loads correctly after a full page reload |
| Preconditions | User has an existing session with at least two messages |
| Test Input | Refresh page; click on the session in the sidebar |
| Expected Output | Full message history displayed in the correct order; no messages missing |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.8 UC-8: Delete Session

**Table 4.15:** TC-08-1: Delete an Existing Session

| Field | Details |
|---|---|
| Test Case ID | TC-08-1 |
| Use Case | UC-8: Delete Session |
| Objective | Verify that deleting a session removes it from the sidebar and from MongoDB |
| Preconditions | User has at least one existing session |
| Test Input | Click the delete icon next to a session |
| Expected Output | Session disappears from sidebar; GET /sessions no longer returns it; MongoDB record deleted |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.16:** TC-08-2: Delete with Invalid Session ID

| Field | Details |
|---|---|
| Test Case ID | TC-08-2 |
| Use Case | UC-8: Delete Session |
| Objective | Verify that a DELETE request with a non-existent session ID is handled gracefully |
| Preconditions | User authenticated |
| Test Input | DELETE /sessions/`nonexistent-id-12345` |
| Expected Output | HTTP 404; error message returned; no crash |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.9 UC-9: Admin Login

**Table 4.17:** TC-09-1: Admin Login with Valid Credentials

| Field | Details |
|---|---|
| Test Case ID | TC-09-1 |
| Use Case | UC-9: Admin Login |
| Objective | Verify that a user with admin role can access the admin dashboard |
| Preconditions | Admin account exists in MongoDB with `role: "admin"` |
| Test Input | Admin username and password on the /admin/login page |
| Expected Output | Login succeeds; admin dashboard rendered; access token contains admin role |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.18:** TC-09-2: Regular User Cannot Access Admin Routes

| Field | Details |
|---|---|
| Test Case ID | TC-09-2 |
| Use Case | UC-9: Admin Login |
| Objective | Verify that a regular user token is rejected on admin-only endpoints |
| Preconditions | Logged in as a non-admin user; access token obtained |
| Test Input | POST /pipeline/ingest-sync with non-admin token in Authorization header |
| Expected Output | HTTP 403 Forbidden; access denied message |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.10 UC-10: Ingest PDF Documents

**Table 4.19:** TC-10-1: Successful PDF Ingestion

| Field | Details |
|---|---|
| Test Case ID | TC-10-1 |
| Use Case | UC-10: Ingest PDF Documents |
| Objective | Verify that a valid PDF placed in the datasets folder is indexed into ChromaDB |
| Preconditions | Admin logged in; at least one PDF in `backend/datasets/`; ChromaDB running |
| Test Input | Click “Ingest PDFs” on the admin panel (POST /pipeline/ingest-sync) |
| Expected Output | HTTP 200; response body shows document count, chunk count, and elapsed time; queries against the indexed content now return relevant results |
| Actual Result | Pass |
| Status | **Pass** |

**Table 4.20:** TC-10-2: Ingestion with Empty Datasets Folder

| Field | Details |
|---|---|
| Test Case ID | TC-10-2 |
| Use Case | UC-10: Ingest PDF Documents |
| Objective | Verify that the ingestion endpoint handles an empty folder without crashing |
| Preconditions | Admin logged in; `backend/datasets/` folder is empty |
| Test Input | POST /pipeline/ingest-sync |
| Expected Output | HTTP 200; response body indicates 0 documents processed; no error |
| Actual Result | Pass |
| Status | **Pass** |

### 4.3.11 Test Summary

Table 4.21 gives a consolidated view of every test case run and the result.

**Table 4.21:** Test Execution Summary

| TC ID | Use Case | Objective (brief) | Status |
|---|---|---|---|
| TC-01-1 | Register Account | Valid registration succeeds | **Pass** |
| TC-01-2 | Register Account | Duplicate email rejected | **Pass** |
| TC-02-1 | Login | Correct credentials accepted | **Pass** |
| TC-02-2 | Login | Wrong password rejected | **Pass** |
| TC-03-1 | Submit Query | English query returns English answer | **Pass** |

*(Table 4.21 continued)*

| TC ID | Use Case | Objective (brief) | Status |
|---|---|---|---|
| TC-03-2 | Submit Query | Urdu query returns Urdu answer | **Pass** |
| TC-03-3 | Submit Query | Roman Urdu query handled correctly | **Pass** |
| TC-03-4 | Submit Query | Empty vector store handled gracefully | **Pass** |
| TC-04-1 | Voice Input | Speech transcribed to query box | **Pass** |
| TC-04-2 | Voice Input | Mic denial handled without crash | **Pass** |
| TC-05-1 | Voice Output | Response read aloud via TTS | **Pass** |
| TC-06-1 | Create Session | New session created and shown | **Pass** |
| TC-07-1 | Load Sessions | History reloads correctly after refresh | **Pass** |
| TC-08-1 | Delete Session | Session removed from DB and UI | **Pass** |
| TC-08-2 | Delete Session | Invalid ID returns 404 | **Pass** |
| TC-09-1 | Admin Login | Admin accesses dashboard | **Pass** |
| TC-09-2 | Admin Login | Regular user blocked from admin routes | **Pass** |
| TC-10-1 | PDF Ingestion | PDF indexed into ChromaDB | **Pass** |
| TC-10-2 | PDF Ingestion | Empty folder returns zero docs, no crash | **Pass** |

All 19 test cases passed. The core RAG flow — query in, response out, same language — worked correctly in all three languages. The JWT lifecycle behaved as designed: expired access tokens were transparently renewed from the refresh cookie without the user noticing. Role enforcement held up: a non-admin token was correctly rejected on the admin ingestion endpoint. The ingestion pipeline handled both the happy path and the empty-folder edge case without errors. There were no unexpected crashes or unhandled exceptions observed during any of the test runs.

# BIBLIOGRAPHY


[1] Ada Health GmbH, "Ada – your personal health guide." https://ada.com, 2024. Accessed: 2025.

[2] K. Singhal *et al.*, "Large language models encode clinical knowledge," *Nature*, vol. 620, pp. 172–180, 2023.

[3] S. Ramírez, "Fastapi – modern, fast web framework for building apis with python." https://fastapi.tiangolo.com, 2018. Accessed: 2025.

[4] MongoDB, Inc., "Mongodb: The developer data platform." https://www.mongodb.com, 2024. Accessed: 2025.

[5] Chroma, "Chromadb: The ai-native open-source embedding database." https://www.trychroma.com, 2023. Accessed: 2025.

[6] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," 2019.

[7] Groq, Inc., "Groq cloud – llama 3.3-70b inference api." https://groq.com, 2024. Accessed: 2025.

[8] Meta Open Source, "React 19 – the library for web and native user interfaces." https://react.dev, 2024. Accessed: 2025.

[9] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.